



Generating and Solving Real World Logic Problems

A dissertation submitted in partial fulfilment of

the requirements for the degree of

Bachelor of Science in Computer Science

in

The Queen's University of Belfast

by

Conall Mone

15/4/2024

CSC3002 – COMPUTER SCIENCE PROJECT

Dissertation Cover Sheet

Student Name: Conall Mone

Student Number: 40285079

Project Title: Generating and Solving Puzzles

Supervisor: Iain Styles

Declaration of Academic Integrity

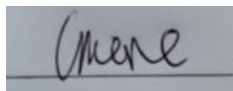
Before submitting your dissertation please check that the submission:

1. Has a full bibliography attached laid out according to the guidelines specified in the Student Project Handbook.
2. Contains full acknowledgement of all secondary sources used (paper-based and electronic)
3. Does not exceed the specified page limit
4. Is clearly presented and proof-read
5. Is submitted on, or before, the specified or agreed due date. Late submissions will only be accepted in exceptional circumstances or where a deferment has been granted in advance.

By submitting your dissertation, you declare that you have completed the tutorial on plagiarism at <http://www.qub.ac.uk/cite2write/introduction5.html> and are aware that it is an academic offence to plagiarise. You declare that the submission is your own original work. No part of it has been submitted for any other assignment and you have acknowledged all written and electronic sources used.

6. If selected as an exemplar, I agree to allow my dissertation to be used as a sample for future students.

Student's signature:



Date of submission: 15/4/24

Abstract.....	5
Introduction.....	5
Problem Area.....	5
Background on Sudoku.....	6
Background on Einstein Puzzle.....	7
Research Objectives.....	7
Literature Review.....	8
Backtracking Algorithm.....	8
Backtracking in Sudoku and Einstein Puzzle Solving.....	9
Natural Language Processing.....	9
SpaCy NLP in Einstein Puzzle Solving.....	10
Fisher-Yates Shuffle Algorithm.....	11
Sudoku Ruleset.....	11
Einstein Puzzle Ruleset.....	11
System Requirements.....	12
Specification.....	12
Methodology.....	14
Development Process.....	14
Technology Stack.....	14
System Architecture.....	15
Implementation of Sudoku Generation and Solving.....	17
Implementation of Einstein Generation and Solving.....	19
Implementation of Front End.....	23
Solving Algorithm.....	26
Testing and Evaluation.....	27
Overview.....	27
Testing Sudoku Problem.....	28
Testing Einstein Problem.....	31
CI Testing.....	35
Evaluation.....	36
Results.....	37
Performance Analysis.....	37
Sudoku Performance Results.....	37
Sudoku Results Analysis.....	37
Einstein Performance Results.....	40
Einstein Results Analysis.....	40
Comparative Analysis.....	42
Discussion.....	44
Interpretation of Results.....	44
Challenges and Limitations.....	44
Implications of Practice and Research.....	45
Conclusion.....	46
Summary.....	46
Recommendations.....	47

Reflection.....	47
Overview.....	47
Reflection on Research Objectives.....	48
Reflection on Requirements.....	49
References.....	50
Appendices.....	50
User Manual.....	50
Appendix A.....	52
Appendix B.....	53
Appendix C.....	54
Appendix D.....	55
Appendix E.....	57
Appendix F.....	60

Abstract

This paper explores the development of a versatile system for generating and solving digital logic puzzles, focusing on Sudoku and Einstein's 5 house logic problem. Challenges such as maintaining puzzle integrity, creating user-friendly interfaces, and adapting solving algorithms to different puzzle types are addressed. The system aims to provide engaging experiences for players while ensuring solvability and accessibility across various difficulty levels. By combining technical expertise and user-centred design principles, the system offers a seamless and immersive puzzle-solving experience. The lessons learned from this endeavour contribute to the ongoing evolution of digital puzzle-solving, enriching the puzzle-solving community.

Introduction

Problem Area

In this paper, I embark on the challenge of constructing a resilient and adaptable system for puzzle generation and resolution in the digital realm. The puzzles chosen for implementation encompass Sudoku and Einstein's 5 house logic problem, offering a diverse spectrum of puzzle complexity: Sudoku, characterised by its constrained structure, and Einstein's puzzle, which relies heavily on linguistic reasoning.

Let's begin by setting out the puzzles and their rules before delving into the methods for solving them.

Sudoku: Sudoku is a number-placement puzzle where the objective is to fill a 9×9 grid with digits so that each column, each row, and each of the nine 3×3 subgrids contain all of the digits from 1 to 9. The puzzle starts with some cells already filled in, and the player's task is to fill in the remaining cells, ensuring that the solution is unique and adheres to the rules.

Einstein's 5 house logic problem: This logic puzzle involves a scenario where there are five houses of different colours, with a different nationality living in each house, owning different pets, drinking different beverages, and smoking different brands of cigarettes. The challenge is to deduce the properties of each house using a set of clues provided, resulting in a single valid solution.

Transitioning traditional paper-based puzzles into digital formats introduces unique challenges, particularly in ensuring the integrity of the puzzles and providing equitable gameplay experiences. For puzzles like Sudoku, where multiple valid configurations are possible, the system must derive a valid solution from the array of possibilities that change with each number added or removed. This balancing act necessitates not only stimulating yet solvable experiences for players but also ensuring accessibility and captivation across various difficulty levels.

Crafting intuitive and user-friendly interfaces poses another formidable challenge. Clear presentation of information and seamless feature identification are paramount for enhancing the overall user experience. The efficacy of the interface directly influences users' ability to interact fluidly with the digital puzzles, shaping their enjoyment and immersion in the gaming experience.

Each puzzle type introduces its own distinct set of challenges, dictated by its unique ruleset and generation process. Despite these variations, maintaining consistency in the application of solving algorithms across different puzzle types is crucial for ensuring a cohesive and seamless experience for players.

Formally, the problem at hand is to develop a system capable of generating and solving puzzles that can be formulated as constraint satisfaction problems. This involves creating a framework that can handle the constraints inherent in each puzzle type while providing efficient and engaging solutions for players.

Background on Sudoku

The problem with this puzzle is the strict and coherent rules that must be followed for generating and solving the problem. Sudoku has several different grids and sizes so I must design code to facilitate the multiple types of grids. The rules are for example on a 9x9 grid must be filled in with numbers from 1-9 with no repeated numbers in each line, horizontally or vertically, to complicate it further the grid is separated into blocks (e.g., a 9x9 grid has 9 3x3 blocks or a 16x16 grid has 16 4x4 blocks) and no repeated number can appear inside the same block.

The approach to this problem will be to design a system using strict constraint handling that will follow the rules to generate and solve the problem then constricting the max number equal to the width or height of the grid and then finding the current numbers on the grid to develop constraints

around those to begin solving the puzzle, each number placed on the grid creates a new constraint and will repeat this until the full grid is solved. Generating the puzzle will involve the same constraint handling for the rules and randomly input numbers across the grid. The expected result for this problem could have several solutions that all adhere to the rule set provided.

Background on Einstein Puzzle

The problem with this puzzle is that it requires the understanding of language to solve the puzzle. Einstein's puzzle is laid out by describing 5 different coloured houses and explaining that 5 different people live in these houses and answering a question such as "Who owns the fish" then provides a list of clues to help solve this puzzle. The approach to solving this riddle is by making a grid with a top row for each house numbered 1 - 5 and in each column list the main attributes such as nationality, colour of house and type of pet. By using the clues to fill out the grid the answer will be easy to see for any player for example "The German owns the fish".

The solution to this problem is that I will randomly generate a list of 5 houses and assign randomly the 5 attributes. The next step is generating the clues. To keep things simple I will use the houses as starting points and create phrases revealing an attribute about each owner and an attribute about the owner's neighbour(s), this will provide an easy spread of phrases for each house. For solving the puzzle, I will need to generate constraints similar to sudoku but using the clues to create the rules for this problem and assign according to the attributes to the correct house to eventually solve the initial question asked. Similarly, to sudoku it requires using constraints to fill out a grid, but this adds more complexity by using language to create said constraints. Same as before every answer placed in the grid will create a new constraint to help solve the puzzle.

Research Objectives

1. To design and develop a user-friendly web application capable of generating and solving Sudoku puzzles: This objective focuses on the creation of a digital platform that provides users with an intuitive interface for generating, solving, and interacting with Sudoku puzzles.
2. To implement efficient solving algorithms for Sudoku and Einstein puzzles: This objective involves developing and implementing algorithms capable of efficiently solving both Sudoku and Einstein puzzles within the web application, ensuring accuracy and speed in puzzle resolution.

3. To evaluate the performance and usability of the web application: This objective entails conducting usability testing and performance evaluations to assess the effectiveness, efficiency, and user satisfaction of the developed web application in generating and solving Sudoku and Einstein puzzles.
4. To compare the effectiveness of different solving algorithms: This objective involves comparing the performance of various solving algorithms implemented within the web application to determine their efficacy in solving Sudoku and Einstein puzzles under different scenarios and constraints.
5. To explore potential applications and implications of the web application beyond entertainment: This objective aims to investigate the potential educational, research, and practical applications of the developed web application beyond mere entertainment, such as in educational settings, research endeavours, and collaborative problem-solving initiatives.
6. To identify challenges and limitations in developing puzzle-solving web applications: This objective involves identifying and addressing challenges and limitations encountered during the development process, including technical constraints, algorithmic complexities, and user interface design considerations.

Literature Review

Backtracking Algorithm

Backtracking is a very popular and useful technique already used in a variety of puzzles for example its most notable reference is for the 8 queen chess puzzle. [1] Lijo, V. P., and Jasmin T. Jose. "Solving N-Queen Problem by Prediction." *Int. J. Comput. Sci. Inf. Technol* 6 (2015): 3844-3848. It is an extremely useful algorithm and the completion is done incrementally, by a sequence of candidate extension steps. Conceptually, the partial candidates are represented as the nodes of a tree structure, the potential search tree. Each partial candidate is the parent of the candidates that differ from it by a single extension step; the leaves of the tree are the partial candidates that cannot be extended any further. In practise it works by finding the first valid answer in an empty grid then traversing to the next leaf until either the puzzle is solved or none of the possible answers work for the current leaf, in

this scenario the algorithm will recursively go back to the previous leaf and check the next valid answer, eventually the grid will be filled with only valid answers and the puzzle is solved.

Backtracking in Sudoku and Einstein Puzzle Solving

In this section, I focus on the implementation of the backtracking algorithm for solving Sudoku puzzles, drawing upon existing research and methodologies. The backtracking technique has been widely recognized for its efficacy in tackling constraint satisfaction problems, such as Sudoku, by systematically exploring potential solutions and backtracking when encountering dead ends.

One notable study by Maji, Arnab Kumar, and Rajat Kumar Pal, presented in the 2014 IEEE International Advance Computing Conference (IACC), introduced a Sudoku solver leveraging mini grid-based backtracking [2] Maji, Arnab Kumar, and Rajat Kumar Pal. "Sudoku solver using minigrid based backtracking." *2014 IEEE International Advance Computing Conference (IACC)*. IEEE, 2014. This approach provided a solid foundation for my implementation, offering insights into the application of backtracking algorithms specifically tailored for Sudoku puzzles.

To adapt this technique for solving the Einstein puzzle as well, I made slight modifications to the existing code, ensuring it accommodated the unique puzzle requirements while retaining the robustness of backtracking algorithms. By integrating language processing techniques, initial solved phrases were generated on the grid, with the backtracking algorithm then used to solve the remaining unanswered phrases.

In Sudoku solving, the backtracking algorithm iteratively fills empty grid cells with potential values, ensuring adherence to the puzzle's rules. If a dead end is reached, the algorithm backtracks to explore alternative solutions. This process grows increasingly complex as the grid fills out, especially with larger grid sizes, posing computational challenges.

Overall, the backtracking algorithm serves as a fundamental tool in solving Sudoku and Einstein puzzles, providing a systematic and rigorous approach to puzzle resolution. Its adaptability and efficiency make it a cornerstone of our solving algorithm, demonstrating its applicability across a diverse range of logic problems. The collaborative use of backtracking and language processing underscores the versatility and effectiveness of our approach in tackling complex puzzle-solving tasks.

Natural Language Processing.

For the Einstein puzzle, the utilisation of Natural Language Processing (NLP) was imperative. This required the selection of a suitable library, and after careful consideration, Python emerged as the preferred language due to its extensive ecosystem of AI-related packages. As noted by Chowdhary, KR1442, and K. R. Chowdhary in their work on "Natural language processing" in "Fundamentals of Artificial Intelligence", [3] Chowdhary, KR1442, and K. R. Chowdhary. "Natural language processing."

Fundamentals of artificial intelligence (2020): 603-649. Python offers a plethora of benefits in the realm of NLP.

Among the available Python libraries for NLP, spaCy was chosen for its robust capabilities and user-friendly features. spaCy provides a comprehensive suite of tools for text processing, including tokenization, part-of-speech tagging, entity recognition, and dependency parsing. Its efficiency and accuracy make it well-suited for the linguistic requirements of the Einstein puzzle, particularly in generating initial solved phrases on the grid.

By leveraging spaCy within the puzzle-solving framework, I planned to harness the power of NLP to process textual clues and generate meaningful solutions for the Einstein puzzle. The integration of NLP techniques alongside the backtracking algorithm exemplifies the interdisciplinary approach taken in developing my puzzle-solving system, showcasing the synergy between linguistic analysis and computational logic in tackling complex problem-solving tasks.

SpaCy NLP in Einstein Puzzle Solving

For the Einstein puzzle, the application of Natural Language Processing (NLP) entails parsing textual clues to derive meaningful constraints for solving the logic problem. spaCy's advanced linguistic processing capabilities enable the extraction of key information from the puzzle's textual clues, such as the nationality, colour, and pet of each person. By employing techniques such as named entity recognition and dependency parsing, spaCy facilitates the identification and organisation of these crucial elements into a structured format suitable for puzzle-solving algorithms. Furthermore, spaCy's support for custom rule-based matching enables the formulation of specific constraints based on the linguistic patterns observed in the puzzle clues. This approach not only enhances the accuracy of the generated solutions but also streamlines the puzzle-solving process by automating the extraction and interpretation of textual information. Overall, the integration of NLP techniques, particularly through the utilisation of spaCy, significantly enhances the efficiency and effectiveness of solving the Einstein puzzle within the digital framework.

Fisher-Yates Shuffle Algorithm

The Fisher-Yates shuffle algorithm is a widely-used method for generating random permutations of a finite sequence. Its simplicity and effectiveness make it an ideal choice for shuffling lists in various applications, including the generation of puzzles like Sudoku and Einstein's logic problem. By employing the Fisher-Yates shuffle algorithm as a list extension, the system gains the capability to shuffle lists of varying lengths and contents efficiently. This extension serves as a foundational component in the puzzle generation process, ensuring randomness and unpredictability essential for creating engaging and diverse puzzle experiences. [4] Fisher, Ronald. "Fisher-Yates shuffle."

Sudoku Ruleset

Sudoku puzzles adhere to a strict set of rules governing the placement of numbers within a grid. In a standard Sudoku grid, consisting of nine rows, nine columns, and nine 3x3 subgrids, each cell must be filled with a single digit from 1 to 9. The primary rule of Sudoku dictates that each row, column, and 3x3 subgrid must contain unique numbers, with no repetition of digits. This constraint ensures that every row, column, and subgrid is filled with distinct numbers, thereby creating a coherent and solvable puzzle. Additionally, Sudoku puzzles often include a subset of pre-filled cells, known as "givens" or "clues," which provide initial information to guide the solving process. These givens must adhere to the rules of Sudoku and serve as the starting point for logical deduction and elimination strategies employed by solvers. By adhering to these fundamental rules, Sudoku puzzles maintain their characteristic challenge and appeal, requiring players to apply deductive reasoning and critical thinking skills to achieve a unique and valid solution. [5] Delahaye, Jean-Paul. "The science behind Sudoku." *Scientific American* 294.6 (2006): 80-87.

Einstein Puzzle Ruleset

The Einstein puzzle, also known as the Einstein's Five Houses puzzle, is a logic problem that requires deducing a unique set of attributes for each of five houses arranged in a linear sequence. The puzzle provides a series of textual clues describing various characteristics of the houses and their occupants, such as colour, nationality, beverage preference, cigar brand, and pet type. The ruleset for the Einstein puzzle revolves around deducing the specific configurations of these attributes for each house based on the provided clues, subject to constraints such as:

- Each house is occupied by a person of a different nationality and no person has the same as another.

- Each house has a distinct colour, and no two houses share the same colour.
- Each person has unique preferences for beverages, cigar brands, and pet types, and no two persons share the same preference or pet.
- Additional constraints provided in the clues guide the determination of specific relationships between attributes, such as the nationality of the person who drinks a particular beverage, or the colour of the house next to the one where a certain pet is kept.

These rules collectively establish the framework within which the puzzle must be solved, guiding the logical deductions necessary to determine the unique characteristics of each house and its occupant. Solving the Einstein puzzle entails applying deductive reasoning and logical inference to derive the precise arrangement of attributes that satisfies all given conditions, ultimately leading to the solution of the puzzle. [6] Patterson, Mike C., Bob Harmel, and Dan Friesen. "A spreadsheet solution to Einstein's Riddle." *International Journal of Technology, Policy and Management* 7.1 (2007): 49-67.

To simplify my puzzle design I only used the attributes Name, House colour, Pet and Nationality.

System Requirements

Specification

The puzzle-solving platform designed in this document aims to provide a versatile system capable of generating and solving digital logic puzzles, with a primary focus on Sudoku and Einstein 5 house logic problems. This platform addresses challenges related to puzzle integrity, user interface design, and algorithm adaptation. By leveraging a robust technology stack and adhering to user centred design principles, the platform offers an engaging and seamless puzzle-solving experience.

Function Definitions

1. Generate the puzzle: The system should be able to generate Sudoku puzzles and Einstein's 5 house puzzle of varying difficulty levels
2. Solve Puzzle: The platform should provide functionality to solve generated puzzles and allow users to enter inputs to be validated and accounted for when solving. With generated hints also to be accounted for.
3. User Interface: the user interface should be intuitive, responsive, and visually appealing, facilitating easy interaction with the puzzle solving algorithm.

4. Text Analysis: the system must utilise SpaCy NLP to analyse textual input generated from sentence based puzzles identifying key entities, phrases and relationships essential for puzzle solving.

Requirements

Functional

- Generate Sudoku and Einstein Puzzles of varying difficulty
- Allow for user input for solving the puzzle to further increase difficulty when solving.
- Offer hint options to increase complexity of the solving algorithm and aid users when necessary.
- Solve Puzzles with time taken to solve puzzles displayed on screen for analysis.
- User interface displaying all necessary information required

Non-Functional

- Performance: The system should generate and solve the puzzles efficiently, with minimal latency even under peak workload conditions
- Scalability: The platform should be scalable to accommodate a growing user base and increasing computational demands.
- Accessibility: The user interface should be accessible to users with intuitive design.
- Reliability: The system should be robust and reliable, minimising downtime and errors during puzzle generation and solving process

Interface Description

1. Backend Interface: the backend will expose RESTful APIs for communication with the frontend, enabling functionalities such as puzzle generation and solving.
2. Frontend Interface: The frontend will interact with the backend through http requests, displaying puzzle interfaces and receiving solutions for display to users. Along with sending back user input requests.

User Interface Requirements

1. Responsive Design: The user interface should adapt seamlessly to various screen sizes and devices, ensuring optimal user experience across platforms.
2. Intuitive Controls: Controls for interacting with puzzles should be intuitive and easy to use, catering to users of all skill levels.

User Characteristics

The target users for the puzzle-solving platform include puzzle enthusiasts, students, educators, and anyone interested in challenging their logical reasoning skills. Users may vary in age, technical proficiency, and familiarity with puzzle-solving techniques.

Methodology

Development Process

Technology Stack

In selecting the technology stack for the puzzle solving platform, several key considerations were taken into account to ensure scalability, efficiency, and functionality. The chosen stack was carefully curated to meet the specific requirements of the project while leveraging industry-standard tools and frameworks.

Backend: The backend of the platform relies on the robustness and versatility of the .NET C# Web application framework. This choice was driven by the need for a solid foundation capable of implementing complex algorithms and managing puzzle generation and solving processes efficiently. The strong type system and extensive libraries of .NET contribute to the reliability and performance of the backend, while the ease of use and setup of controller endpoints streamline development processes.

Unit Testing: XUnit was integrated into the technology stack to facilitate comprehensive unit testing of backend functionalities. Its adaptability and reliability make it an ideal choice for ensuring the accuracy and stability of backend components, particularly within the .NET web application environment.

Continuous Integration Testing: GitLab CI testing plays a vital role in ensuring the seamless integration of new features into the platform through automated validation of code changes. By employing GitLab CI testing for continuous integration testing across the backend, frontend, and Python web service components, the platform maintains a high level of reliability and stability.

Frontend: React JS was selected for the frontend to capitalise on its flexibility, speed, and modular architecture. The interactive user interface, built using React components, ensures a responsive and

engaging user experience. The separation of concerns between the frontend and backend allows for independent development and easier maintenance, enhancing the overall agility and scalability of the platform.

Natural Language Processing (NLP): SpaCy, a powerful and efficient library for NLP tasks, was integrated into the technology stack to enhance the platform's capabilities in text analysis. By leveraging SpaCy's pre-trained models and functionalities such as tokenization, part-of-speech tagging, named entity recognition, and dependency parsing, the platform gains the ability to analyse textual inputs from users effectively. This functionality is crucial for understanding puzzle descriptions, user instructions, and any textual input related to the puzzle-solving process, thereby enriching the overall user experience.

In summary, the technology stack was chosen based on the specific requirements of the project, with each component selected to contribute to the scalability, efficiency, and functionality of the puzzle solving platform.

System Architecture

The architecture of the puzzle-solving platform encompasses a frontend, backend, and a single web service. The backend, developed in .NET, features endpoints specifically tailored for handling Einstein and Sudoku puzzles. These endpoints enable puzzle generation and solving functionalities.

Meanwhile, the frontend serves as the user interface, facilitating user interactions and displaying puzzle-related information. It communicates with the backend by making calls to the designated endpoints and processing the received data accordingly. Additionally, a Python Flask web service is utilised to manage Einstein puzzle generation. This service returns a list of solved phrases, which is essential for initially populating the puzzle grid. Below is a uml diagram designed to show how my

[illegible]

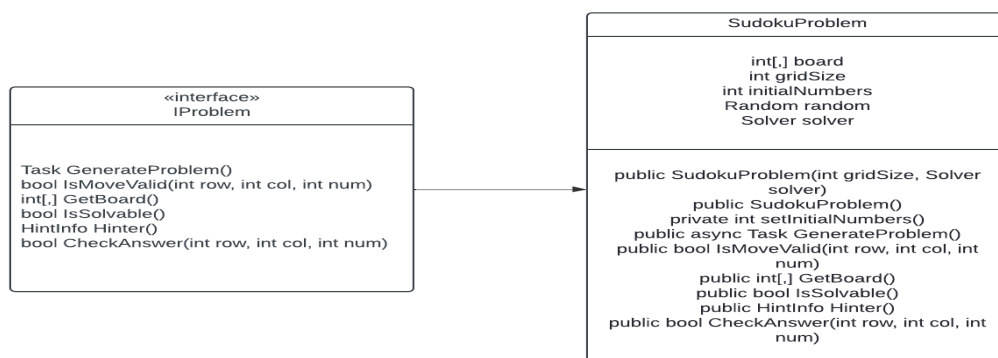
The system architecture, depicted in the accompanying UML diagram, reflects diligent planning and execution, aiming to deliver a dynamic and scalable platform capable of accommodating not only Sudoku and Einstein puzzles but also any potential future puzzle implementations. Key architectural practices include the implementation of an `IProblem` interface, which serves as a parent class for puzzle types such as Einstein and Sudoku. This interface facilitates a modular approach to puzzle handling, allowing for seamless integration of new puzzle types. Furthermore, the architecture emphasises the utilisation of controllers as endpoints within the backend. These controllers play a crucial role in handling incoming requests from the frontend, enabling efficient data exchange and interaction between the various system components.

To manage the user interface and user inputs effectively, a React JS frontend was created. It comprises distinct pages tailored to handle the two puzzle types showcased for puzzle generation

and solving. The interface seamlessly orchestrates all communication with the backend, transmitting essential data and adeptly presenting and managing information as needed.

Implementation of Sudoku Generation and Solving

The implementation of Sudoku within the puzzle-solving platform relies significantly on the meticulous handling and enforcement of strict constraints dictated by the rules of the game. This approach ensures the integrity of the puzzles generated and facilitates the creation of a dynamic system capable of accommodating various puzzle iterations. Central to this strategy is the `IProblem` interface, which encapsulates essential functions that any puzzle within the system must adhere to. These functions, including `Generate`, `IsMoveValid`, `IsSolvable`, `Hint`, and `CheckAnswer`, provide a standardised framework for puzzle generation, validation, hinting, and solution verification. By abstracting these functionalities into a cohesive interface, the platform becomes highly adaptable, enabling the seamless integration of new puzzle types while maintaining consistency and reliability across the board.



To initiate the implementation of Sudoku generation, a child class named `SudokuProblem` was derived from the `IProblem` interface, inheriting essential functions for generation and solving. Each puzzle type introduces its own unique generation steps. In the case of Sudoku, the initial step involves populating a grid with starting numbers. The default grid size is set to 9x9, with users having the option to select from sizes including 2x2, 4x4, 9x9, and 16x16. The initial numbers are calculated by selecting a quarter of the total grid spaces.

Upon determining the number of initial starting numbers, the grid randomly selects a row, column, and number (ranging from 1 to the grid size) to populate. With each number added to the grid, several checks are performed. Firstly, the `IsMoveValid` function, inherited from the `IProblem`

interface, employs the unique set of Sudoku rules to validate the input. If the input fails validation, another random number is selected until a valid answer is found.

After successful validation, the `IsSolvable` function, inherited from `IProblem`, further ensures puzzle integrity by verifying whether the inputted number leads to a solvable Sudoku grid. This process involves creating a copy of the current board state and passing it to the backtracking solver. The solver attempts to solve the puzzle, returning either `true` (indicating successful solution) or `false` (indicating unsolvability). If the puzzle cannot be solved, the function loops through the setup again until a valid configuration is found. This meticulous check guarantees the authenticity of the puzzle, addressing instances where the grid could be populated with valid answers but remained unsolvable in its final state.

Once both checks succeed and all initial numbers are populated, the grid is stored in a board variable, accessible through the various endpoints developed. This meticulous approach guarantees the validity and reliability of the generated Sudoku puzzles.

The hint function operates by identifying all empty grid spaces within the board variable and utilising a shuffle list extension inspired by the Fisher-Yates shuffle algorithm to randomise their order. Subsequently, the function systematically attempts to populate the empty spaces with numbers ranging from 1 to the grid size, employing a similar approach to backtracking. This process continues until the first number is found to pass both the `IsValidMove` and `IsSolvable` checks. Through various stages of manual testing, this iteration of the hint option has proven to be the most effective. Previous iterations, such as randomly selecting a row, column, and number and subsequently checking its validity, yielded subpar results and were consequently refined to meet the current iteration.

One challenge encountered with the hint function was its potential impact on system memory when multiple calls were made successively. To address this issue, a limitation was imposed, restricting users to a maximum of three hints per puzzle. This not only prevents memory abuse but also enhances the efficiency of other functions, resulting in a more game-like experience for users.

The check answer function handles a variable passed from the frontend containing row, column, and number information. Upon retrieval, the number is entered into the backend's board variable, where it undergoes validation checks using `IsValidMove` and `IsSolvable`. Upon successful validation, the

board is updated, and a confirmation message is sent back to the frontend. Conversely, if the validation fails, an error message is returned without altering the board state.

For simplicity and dynamic functionality between components, only valid answers are stored in the board variable. As a result, the frontend grid never displays incorrect answers, with an alternative indicator utilised to differentiate correct and incorrect answers. This streamlined approach ensures a seamless and intuitive user experience while maintaining puzzle integrity.

Similar to puzzle generation, solving the Sudoku grid hinges on the meticulous application of strict and coherent constraint handling. In the realm of Sudoku solving, the backtracking algorithm designed is the most efficient and reliable method. This algorithm navigates through the puzzle grid, systematically filling in missing numbers while adhering to the established Sudoku rules.

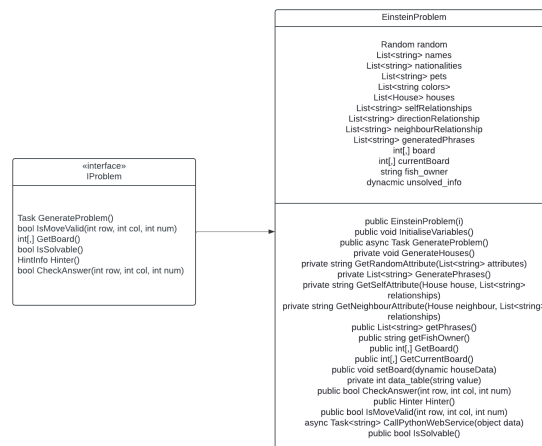
The solver function operates by accepting an instance of the Sudoku problem and leveraging the current state of the board to progress towards a solution. Through the iterative application of the backtracking algorithm, the solver systematically explores possible number placements, backtracking when necessary to correct any inconsistencies or violations of Sudoku constraints. This iterative process continues until a valid solution is found for the entire grid.

By relying solely on the backtracking algorithm for solving, the implementation ensures robust and accurate puzzle resolution, aligning with the high standards of puzzle integrity upheld by the platform.

Implementation of Einstein Generation and Solving

The integration of the Einstein puzzle into the puzzle-solving platform posed a unique challenge due to its reliance on extracting information from worded phrases. Much like Sudoku, the Einstein puzzle is implemented using the `IProblem` interface, employing familiar methods for both puzzle generation and solving. However, unlike Sudoku, the Einstein puzzle demands a sophisticated combination of strict constraint handling and natural language processing to decipher constraints embedded within textual descriptions and apply corresponding rules to the puzzle grid. For simplicity the same question for every puzzle is being “Who owns the fish” as this will assist in generation and solving the puzzle.

This conjoined approach allows the platform to accurately interpret and translate textual clues into actionable puzzle constraints, ensuring that the generated Einstein puzzles adhere to the logic and rules governing the game. By leveraging both constraint handling techniques and natural language processing capabilities, the platform can effectively generate and solve Einstein puzzles, offering users a challenging and intellectually stimulating experience.



The generation process for the Einstein problem begins with the creation of random house variables, each representing a unique dwelling within the puzzle scenario. Essential attributes such as names, nationalities, house colours, and pets are predefined and stored as variables within the system. Utilising these variables, a custom House object is randomly generated. This process involves shuffling the list of names using the Frank-Yates algorithm, iterating through each house in the puzzle scenario, and assigning random attributes. The index of the loop determines the house number, while random selections from the corresponding attribute lists determine the name, nationality, colour, and pet for each house. By creating these House objects, the system ensures accuracy and coherence in generating phrases that reflect the puzzle's constraints and rules.

The creation of phrases follows a systematic approach aimed at generating coherent and informative sentences for each house variable. For every house, three sentences are crafted: one describing an attribute of the house itself, and the other two detailing attributes of its neighbouring houses (if applicable). The structure of these sentences consistently begins with the owner's name, followed by the attribute being revealed. This uniformity in sentence structure aids in simplifying the process of deriving answers from the phrases during puzzle solving. Additionally, to provide further control over phrase generation, three predetermined sentences are included: one specifying who lives in the first house, another indicating the occupant of the last house, and a third disclosing the colour of the

middle house. These fixed phrases serve as anchors for the Python script, facilitating the assignment of values to house variables and ensuring coherence and accuracy in the generated puzzle scenarios.

Following the generation of phrases, they are shuffled and passed to a Python script for assignment. Initially, the script attempts to utilise SpaCy NLP to extract attributes such as colour, person name, animal, and language. However, due to inconsistent accuracy, a more restrictive approach is adopted. This involves searching for names or attributes from a predetermined list. Although this approach does not fully harness the potential of natural language processing, it was chosen due to time constraints and deadlines.

The Python script first focuses on decoding the anchor hard-written phrases, such as "who lives in the first house," "who lives in the last house," and "middle house colour." These values are then assigned to their respective house numbers, with each phrase removed once its assignment is complete. After resolving the anchor phrases, the script proceeds to analyse each remaining phrase word by word to determine the name, attribute, and directional relationship (such as self or neighbour attribute).

During this analysis, a search is conducted to determine if the name or attribute has already been assigned to a house. If so, the value is assigned accordingly, and the script moves on to the next phrase. If not, the phrase is skipped. To determine neighbour relationships, a straightforward method of adding or subtracting 1 from the house number is employed when words such as "right" or "left" are present in the phrase.

The assignment process continues iteratively until all phrases are successfully resolved or until a loop is encountered where no phrases can be assigned. Unassigned phrases are stored in a list, the fish owner is stored as a value and finally, the house data is returned to the .NET application for further processing. This systematic approach ensures accurate assignment of attributes to houses, facilitating the creation of cohesive and solvable Einstein puzzles within the puzzle-solving platform.

Upon receiving the data, the IsSolvable check is initiated to ensure puzzle solvability. In the case of the Einstein puzzle, IsSolvable verifies that the fish owner attribute is not empty or null, and that the number of unassigned pieces of information is zero, indicating that all phrases have been successfully solved by the Python script. Once this check is passed, the board is set by assigning numerical values to each string value returned as a house attribute.

A numerical approach is employed for board assignment, where each attribute type (such as name, nationality, colour, and pet) is assigned a numerical value ranging from 1 to 5, and the house number is indicated by the index of the board length. This numerical representation facilitates seamless integration with the existing solver used for Sudoku and offers a straightforward approach to puzzle solving.

Once each string value is assigned its respective numerical value, they undergo validation with the `IsMoveValid` check. This verification ensures that no value within the same row appears more than once, as each value in the Einstein puzzle is intended to represent a unique attribute. Upon successful validation, the answers are assigned to the board value.

It's important to note that for this puzzle, there are two board values: one containing the solved word phrases returned from the Python script, and another serving as an empty board for user interaction with the check answer and hint function. This distinction enables users to engage with the puzzle-solving process effectively while maintaining the integrity of the puzzle's solution.

The implementation of the check answer function closely mirrors that of Sudoku. A row, column, and number are passed to the function, where the `IsMoveValid` operation ensures the correctness of the input. Check answer then compares the entered value against the solved word phrases board to determine if it matches. If the corresponding space on the solved board is empty, Check answer checks if the move is valid instead. The function returns a response of true or false, and the front end manages all indications of inputted answers. Additionally, there's an additional check answer function for when a user attempts to solve the riddle by selecting the owner they believe owns the fish. This function simply checks if a true or false value is returned.

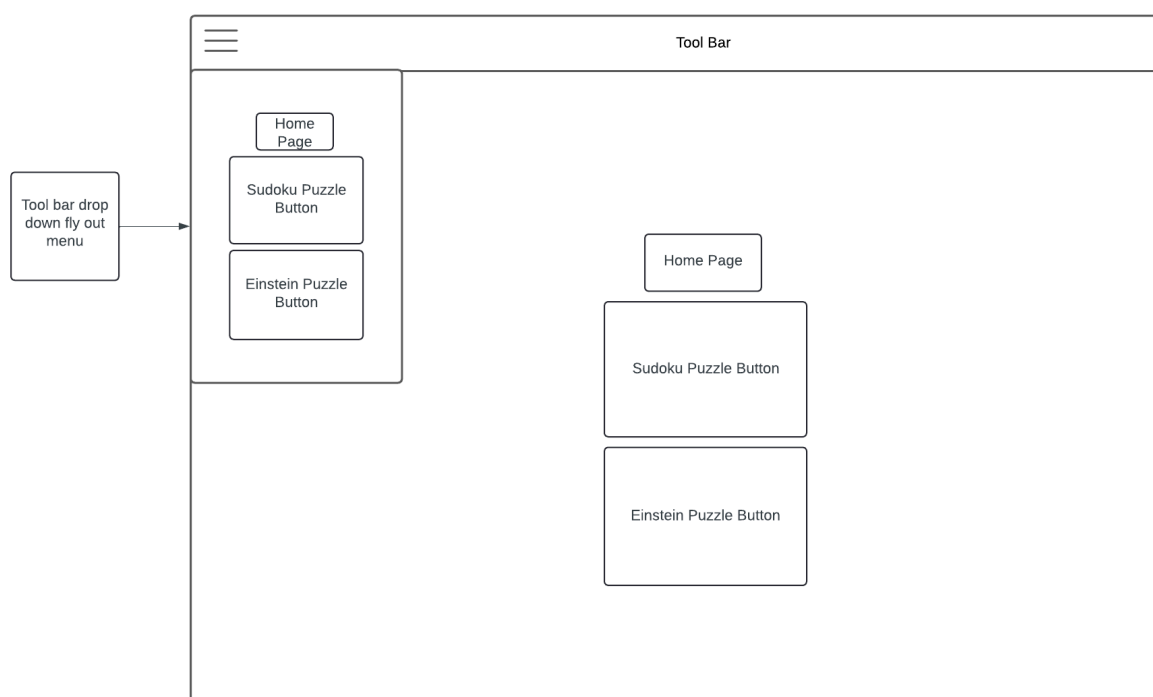
The hint function works by revealing the solved phrases board to the user, with the pet attribute of the fish owner removed to prevent the hint from revealing the answer outright. This approach was chosen due to time constraints and the simplicity of its implementation.

Similarly to Sudoku, the solver function is used to fill in any remaining spaces on the board that the phrases might have missed. This is achieved by passing the board containing the solved phrases to the solver function, which then employs backtracking to determine the correct values for the missing spaces. While this puzzle relies less on the backtracking algorithm compared to Sudoku because most

of the solving starts at the puzzles generation phase when assigning values to houses, it still serves as a valuable tool for efficiently solving the puzzle.

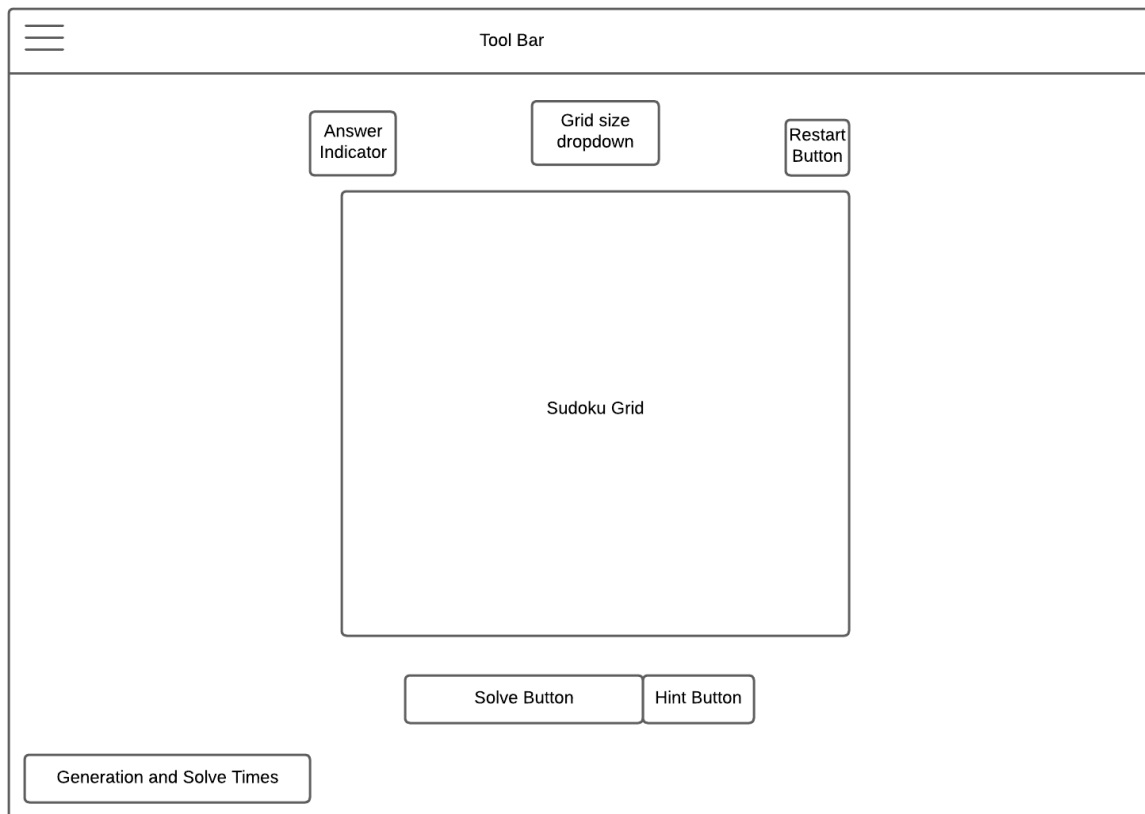
Implementation of Front End

As previously mentioned, React JS served as the framework for designing the front end. To enhance the overall design, additional elements such as Font Awesome icons were integrated. Furthermore, Lucidchart was employed to meticulously plan the layout of the Sudoku and Einstein puzzle pages before actual implementation.



Home Page Design - Appendix B

The home page serves as the initial landing page for users, featuring prominent buttons for accessing Sudoku and Einstein puzzles, each leading to their respective pages. A consistent toolbar is present on every page, facilitating seamless navigation between the home page and puzzle pages. To achieve this, a custom Toolbar component was created and integrated into the App page, ensuring its presence across all pages. This user-friendly design approach enhances efficiency and elevates the overall user experience by enabling smooth transitions between pages.



Sudoku Page Design - Appendix B

The Sudoku page is meticulously planned with the inclusion of essential functionalities encapsulated in buttons: Solve, Hint, and Restart. Each button triggers a specific endpoint when clicked. The Solve button calls the solve endpoint, passing the current state of the grid, and returns a fully solved board along with appropriate success messages. Similarly, the Hint button can be clicked up to three times before displaying an alert message. When Hint is invoked, a hinted number object is returned, allowing the hinted number to be visually distinguished on the board, typically by colouring it differently.

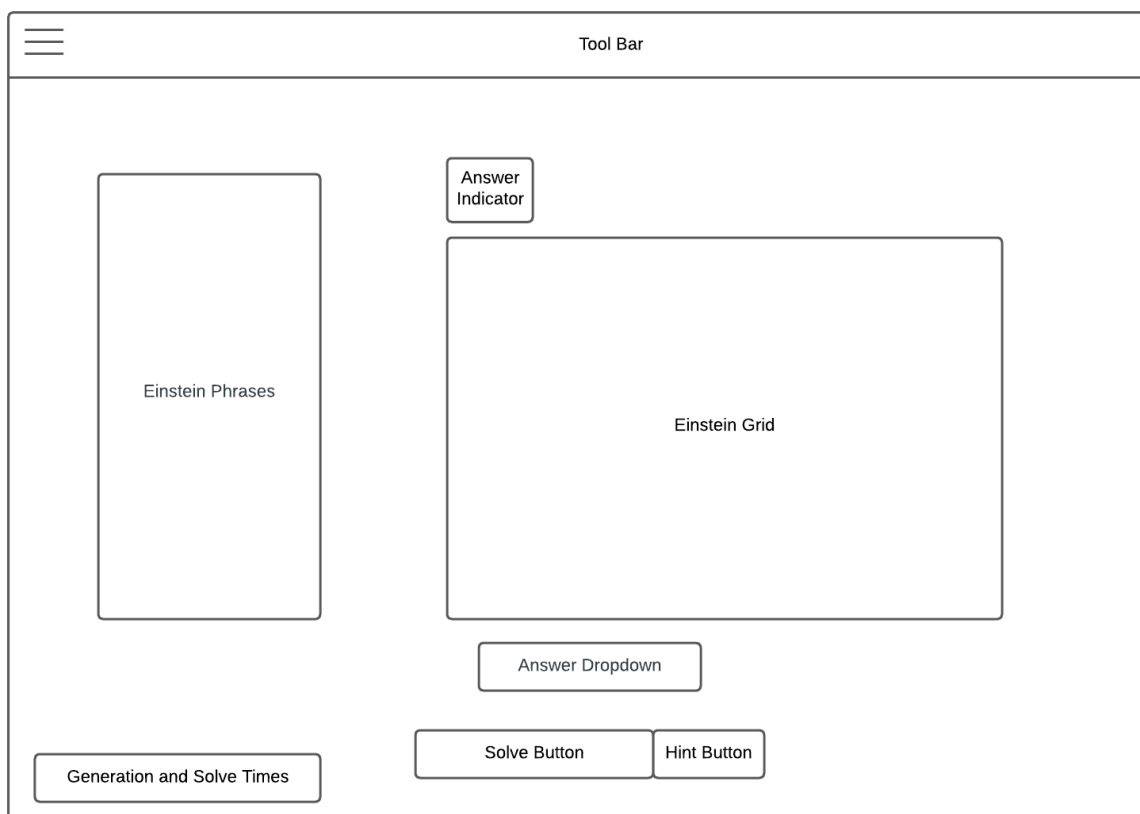
To provide users with a visual representation of available hint points, three icons are displayed, each representing a hint point. These icons decrease in number with each hint used. The Restart button recalls the generate method for the currently selected grid size and resets all values without requiring a page refresh.

The Sudoku grid itself is composed of two custom components: Column and Row. The board length is passed to the Column component, which dynamically generates instances of the Row component. Each Row component contains an input box that facilitates user input. Upon entering a value, the Check Answer endpoint is invoked, updating the state of the board accordingly. If the input is correct,

an answer indicator shows a tick; if incorrect, the board remains unchanged, and a red cross is displayed along with a tooltip indicating the incorrect answer.

Additionally, the grid size dropdown menu allows users to adjust the size of the grid. Selecting a different grid size triggers the Generate endpoint, passing back the selected size and updating the display accordingly (options include 2x2, 4x4, 9x9, and 16x16 grid sizes).

Furthermore, the response times for Generation and Solve are calculated in the front end. These metrics provide insights into the time taken for Sudoku grid generation and solving. They can be utilised to compare the performance and efficiency of different grid sizes, as well as to benchmark against other puzzle-solving platforms and traditional paper-based solving approaches.



Einstein Page Design - Appendix B

The Einstein page follows a layout similar to Sudoku, offering functionalities like the Solve button, answer indicator, and generation and solve times tracking.

Upon page load, the left side of the page displays all the phrases in bullet points along with the question "Who owns the fish?" These phrases, retrieved from the backend, are presented for user

reference. The Einstein grid is constructed using the same Column and Row components. However, in this case, the Row component passes back a dropdown containing all the possible attribute values for each row (name, colour, nationality and pet). The drop down utilises the numbers read from the backend as indexes, effectively converting the numbered grid into a text-based dropdown. Upon selecting a dropdown value, the Check Answer endpoint is called to verify if the selected index is a valid move, updating the answer indicator and grid accordingly, similar to the functionality in Sudoku.

The Hint button retrieves multiple correct answers when clicked and can only be used once. This simplicity aids in saving time and allows for focus on other essential functions. The answer dropdown contains all possible names, and upon selection, a message appears indicating whether the answer was correct or incorrect. Additionally, upon answering, the Restart button appears, allowing users to refresh the page.

When the Solve button is clicked, the grid answers are revealed, and the correct answer is selected from the answer dropdown, accompanied by a success message.

Above the grid, numbered house icons dynamically change colour when the colour attribute is assigned. This adaptive design principle aids users in better distinguishing between the separate houses.

Solving Algorithm

As previously discussed, the solving algorithm employed for Sudoku and Einstein puzzles utilises backtracking as its primary mechanism. This approach allows for flexibility and scalability by accepting an object of the IProblem interface, facilitating the solution of not only existing puzzles but also potential future ones.

Once within the solver class, the algorithm retrieves the current problem instance and accesses the board. Utilising the length of the board as the maximum index, the algorithm iterates through each empty space, attempting to fill it with numbers ranging from 1 to the maximum. At each step, the IsMoveValid method, unique to each problem type, is invoked to validate the current move. If a dead end is encountered, the algorithm backtracks by subtracting one from the index and recursively calling the solver method again. This process repeats until the entire board is successfully filled, resulting in a solved puzzle.

The computational complexity of the algorithm varies depending on factors such as the size and initial state of the board. Larger grid sizes and a greater number of initially filled spaces can significantly increase the time required to solve the puzzle due to the increased number of possible combinations and iterations needed to find the correct solution.

The simplicity of the solver design enhances its versatility and ease of implementation across various problem types and sizes. By adhering to the `IProblem` interface, the solver class can seamlessly adapt to new puzzle implementations without requiring extensive modifications. This modular design allows for rapid development and integration of additional puzzle-solving functionalities, facilitating the expansion of the puzzle-solving platform to accommodate diverse user preferences and requirements.

Moreover, the straightforward nature of the solver algorithm simplifies debugging and maintenance efforts. Its clear structure and logical flow allows developers to identify and address issues efficiently, ensuring the reliability and stability of the puzzle-solving platform. Additionally, the decoupling of the solver functionality from specific puzzle types promotes code reusability, reducing redundancy and enhancing overall system coherence.

Furthermore, the solver's scalability enables it to handle puzzles of varying complexities and sizes, ranging from simple 2x2 grids to more intricate 16x16 grids. This scalability ensures that the puzzle-solving platform can accommodate the evolving needs and preferences of users, providing a rich and engaging experience across a wide range of puzzle types and difficulty levels.

Overall, the solver's simplicity, adaptability, and scalability contribute to the robustness and versatility of the puzzle-solving platform, laying a solid foundation for future enhancements and expansions.

Testing and Evaluation

Overview

Testing the puzzle-solving platform was conducted through a comprehensive approach aimed at ensuring the reliability, functionality, and performance of the system across its various components. Unit testing was implemented for the system's backend, focusing on individual units of code to verify their correctness and robustness. This approach enabled thorough examination of critical

functionalities such as puzzle generation, solving algorithms, and API endpoints, allowing for the detection and resolution of any potential issues early in the development process.

On the frontend, manual testing was performed to assess the user interface design, responsiveness, and overall user experience. Test scenarios were devised to simulate user interactions with the puzzle-solving platform, including navigating through pages, inputting values, and interacting with puzzle-solving features. By manually testing the frontend, developers were able to identify usability issues, layout inconsistencies, and potential bugs, ensuring a seamless and intuitive user experience.

Additionally, continuous integration testing was applied to all three services in the puzzle-solving platform: the backend, frontend, and Einstein puzzle-solving Python script. This involved automating code integration and conducting automated testing to ensure the integrity and functionality of the codebase. While the backend had dedicated test phases, the frontend and Python script underwent manual testing and a build phase. This comprehensive approach upheld the stability and reliability of the platform.

By employing a combination of unit testing, manual testing, and continuous integration testing, the puzzle-solving platform underwent rigorous evaluation to ensure its quality, performance, and adherence to specifications. This multifaceted testing approach contributed to the delivery of a robust, user-friendly, and high-performing platform capable of providing an engaging and seamless puzzle-solving experience for users.

Testing Sudoku Problem

As previously mentioned, an XUnit testing solution was integrated into the puzzle-solving platform backend to facilitate the creation and execution of unit tests for the backend controllers. Specifically, the Sudoku Controller underwent rigorous testing, focusing on key functionalities including Generate, Solve, CheckAnswer, GetCurrentBoard, and Hint. These tests were designed to thoroughly evaluate the platform's ability to handle various scenarios, including error handling, invalid inputs, and correct processing of valid inputs. By testing these core features, the robustness and reliability of the system in managing Sudoku problems were effectively validated.

✓ EinsteinTest (6)	8.2 sec
✓ SudokuTest (23)	18 sec
✓ CheckAnswer_IncorrectAnswer_...	211 ms
✓ CheckAnswer_ReturnsBadRequest	2 ms
✓ Hint_ReturnsBadRequest	< 1 ms
✓ GenerateSudoku_Valid (4)	< 1 ms
✓ GetCurrentBoard_ReturnsBadRequest	< 1 ms
✓ GetCurrentBoard_Valid (4)	< 1 ms
✓ HintSudoku_ValidProblem (4)	1 ms
✓ SolveSudoku_SolvableProblem (..)	17.8 sec
✓ SolveSudoku_ReturnsBadRequest	< 1 ms

Testing Screenshot - Appendix C

The Sudoku controller tests leverage a feature of Xunit, allowing for the passing of a variable representing the grid size to create tests for each of the four available sizes: 2x2, 4x4, 9x9, and 16x16. This comprehensive approach ensures thorough testing of the platform, covering a wide range of scenarios and potential errors. The tests focus on validating inputs, handling invalid inputs, and managing incorrect inputs where applicable. This meticulous testing strategy enhances the platform's robustness and reliability, enabling it to effectively handle Sudoku puzzles of varying complexities.

The subsequent phase of the test plan involved manual testing of the frontend features, as this approach offers the most effective means of verifying the correct response of functions from the frontend to the backend. The test plan employed various techniques, including Equivalence Class Partitioning to cover the main breadth of the system with minimal tests and Boundary Value Analysis to assess the minimum and maximum values, as well as values just beyond these boundaries. This comprehensive testing strategy ensures thorough validation of the system's functionality and responsiveness across different scenarios.

Test Case	Input	Expected Outcome	Pass/Fail
TC1 - change grid size	Select all the options in the grid size dropdown	The grid will change to the corresponding grid size	Pass
TC2 - valid correct	Valid number in grid	Grid updated to reflect answer input. Answer indicator will turn to green tick	Pass

TC3 - valid incorrect	Valid but incorrect number in grid	Grid unchanged. Answer indicator will turn to red cross	Pass
TC4 - invalid input beyond minimum	Enter 0 into grid	Grid unchanged. Answer indicator will turn to red cross	Pass
TC5 - invalid input beyond maximum	Enter a number 1 beyond grid size maximum	Grid unchanged. Answer indicator will turn to red cross	Pass
TC6 - use first hint	Click hint button	Number will display on grid in blue and a hint icon removed	Pass
TC7 - use second hint	Click hint button	Number will display on grid in blue and hint icon removed	Pass
TC9 - use third hint	Click hint button	Number will display on grid in blue and hint icon removed	Pass
TC10 - use fourth hint	Click hint button	Alert appear saying out of hints	Pass
TC11 - Solve button	Click the solve button	Grid will be completed and message will appear	Pass
TC12 - solve with hints	Click the hint button and then the solve button	Grid will be completed and message will appear	Pass
TC13 - manually solve	Manually input the answers to the grid	Grid will be completed and message will appear	Pass
TC14 - refresh button	Click the refresh button	Current selected grid size will refresh a new grid	Pass

Test Case Table - Appendix E

The test plan above was meticulously executed on each of the four Sudoku grid sizes, ensuring comprehensive coverage of the platform's functionality across various scenarios. Each test case was designed to validate different aspects of the platform's behaviour, including input validation, hint usage, solving mechanisms, and grid refresh functionality. By systematically testing these scenarios, the robustness and reliability of the Sudoku solving feature were effectively assessed.

In TC1, the ability to change the grid size was tested, ensuring that the platform accurately reflects the selected size and updates the grid accordingly. Test cases TC2 and TC3 assessed the platform's handling of valid and invalid inputs, respectively, confirming that correct inputs are processed correctly while incorrect inputs trigger appropriate error indicators.

TC4 and TC5 focused on boundary value testing, examining the platform's behaviour when inputs fall beyond the minimum and maximum allowable values. These tests ensure that the platform adequately handles edge cases and provides accurate feedback to users.

The hint functionality was thoroughly tested in TC6, TC7, and TC8, evaluating the platform's ability to provide hints progressively and accurately reflect hint usage on the grid. Additionally, TC9 and TC10 examined the platform's response when the hint limit is reached, verifying that appropriate alerts are triggered.

The solving mechanism was validated in TC11, TC12, and TC13, testing the platform's ability to solve Sudoku puzzles automatically, with and without hint assistance, as well as validating manual solving by users.

Finally, TC14 assessed the platform's grid refresh functionality, ensuring that users can reset the grid to a new state when needed.

Overall, the test plan comprehensively evaluated the Sudoku solving feature, providing assurance of its reliability, accuracy, and user-friendliness.

Testing Einstein Problem

Similarly to Sudoku, the XUnit testing solution included a dedicated file for testing the Einstein controller endpoints within the puzzle-solving platform backend.

✓ EinsteinTest (8)	8.2 sec
✓ CheckAnswer_ReturnsBadRequest	10 ms
✓ CheckFishOwner_ReturnsBadRequest	< 1 ms
✓ CheckFishOwner_ReturnsIncorrect	4.2 sec
✓ GetCurrentBoard_ReturnsBadRequest	< 1 ms
✓ GetPhrases_ReturnsBadRequest	< 1 ms
✓ GetPhrases_Valid	4.1 sec
✓ Hinter_ReturnsBadRequest	< 1 ms
✓ SolveEinstein_ReturnBadRequest	< 1 ms

Testing Screenshot - Appendix C

The testing process primarily focuses on thoroughly assessing the error handling capabilities of each function, in addition to covering the essential functionalities shared with Sudoku, such as solve, check answer, get current board, and hint. Moreover, additional tests were conducted specifically for retrieving the fish owner and phrases associated with the Einstein puzzle. These tests comprehensively explore various scenarios, including invalid and incorrect answer responses, ensuring the robustness and reliability of the platform's Einstein puzzle-solving feature. However, due to the involvement of a Python script in the generation process, which couldn't be integrated into the continuous integration (CI) pipeline, certain tests, such as generate, were excluded from the automated testing phase. These excluded tests were planned to be thoroughly evaluated during the manual testing phase to ensure the complete validation of the puzzle-solving platform's functionalities.

Test Case	Input	Expected Outcome	Pass/Fail
TC1 - valid correct colour	Correct answer selected for colour	Grid updated to reflect answer input. Answer indicator will turn to green tick	Pass
TC2 - valid correct name	Correct answer selected for name	Grid updated to reflect answer input. Answer indicator will turn to green tick	Pass
TC3 - valid correct nationality	Correct answer selected for nationality	Grid updated to reflect answer input. Answer indicator will turn to green tick	Pass
TC4 - valid correct pet	Correct answer	Grid updated to reflect answer	Pass

	selected for pet	input. Answer indicator will turn to green tick	
TC5 - incorrect answer for colour	incorrect answer selected for colour	Grid unchanged. Answer indicator will turn to red cross	Pass
TC6 - incorrect name	Incorrect answer selected for name	Grid unchanged. Answer indicator will turn to red cross	Pass
TC7 - incorrect nationality	incorrect answer selected for nationality	Grid unchanged. Answer indicator will turn to red cross	Pass
TC8 - incorrect pet	incorrect answer selected for pet	Grid unchanged. Answer indicator will turn to red cross	Pass
TC9 - first hint option	Click hint button	The grid will display the correct answers on the grid	Pass
TC9 - second hint option	Click the hint button a second time	An alert will appear stating that you one hint is already used	Pass
TC10 - select correct fish owner	Select the correct fish owner from the select owner dropdown	A success message will display and a refresh button	Pass
TC11 - select correct fish owner	Select the incorrect fish owner from the select owner dropdown	A failed message will display and a refresh button	Pass
TC12 - house colour change	Select a valid house colour answer	The house icon above the selected column will change to the selected colour	Pass
TC13 - solve button	Click the solve button	The grid will populate and the correct fish owner will display with success message	Pass

TC14 - solve button with hints	Click the hint button then the solve button	The grid will populate and the correct fish owner will display with success message	Pass
TC15 - refresh button	Answer the fish owner dropdown then click the refresh button	The page will refresh	Pass

Test Case Table - Appendix E

The test plan outlined above meticulously evaluates the functionality of the Einstein puzzle-solving feature across various scenarios, ensuring thorough coverage and validation of the platform's behaviour.

TC1 to TC8 cover both correct and incorrect dropdown selections for attributes such as name, colour, nationality, and pet, verifying that the platform triggers appropriate error or success indicators accordingly. These tests are highly effective in assessing the platform's ability to handle user inputs and provide accurate feedback, ensuring that users can interact with the puzzle-solving feature intuitively.

TC8 and TC9 specifically address the hint option, ensuring that the platform accurately provides hints and reflects the usage of hints on the grid. These tests effectively evaluate the hint functionality,

TC10 and TC11 focus on checking the correctness of the selected fish owner, validating the platform's response to both correct and incorrect selections. These tests are crucial in ensuring the accuracy of the puzzle-solving process, providing users with clear feedback on their choices and guiding them towards the correct solution effectively.

TC12 assesses the front-end design's ability to adapt colours when a colour is successfully selected for a house on the grid, ensuring a visually cohesive and intuitive user experience. This test evaluates the effectiveness of the platform's visual presentation, enhancing user engagement and comprehension of the puzzle-solving process.

TC13 and TC14 thoroughly assess the functionality of the solving button, both with and without hints applied, to ensure the platform accurately solves the puzzle and provides appropriate feedback to the user. These tests play a critical role in validating the core functionality of the puzzle-solving

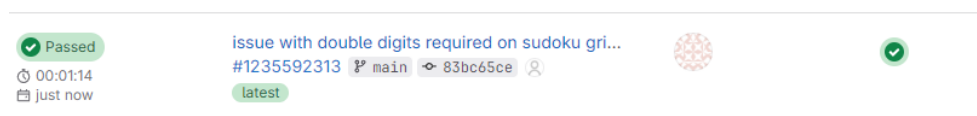
feature, establishing users' confidence in the platform's ability to deliver accurate solutions and timely feedback on their progress. Notably, an issue concerning how the solver responds after an error was rectified by creating a copy of the solved board, preventing inadvertent overwriting. Such issues underscore the importance of meticulous manual testing, as they might not have been discovered otherwise.

Finally, TC15 evaluates the refresh button, verifying that users can reset the grid to a new state when needed, maintaining the platform's usability and responsiveness. This test ensures that users can easily manage their puzzle-solving experience, enhancing overall satisfaction and usability.

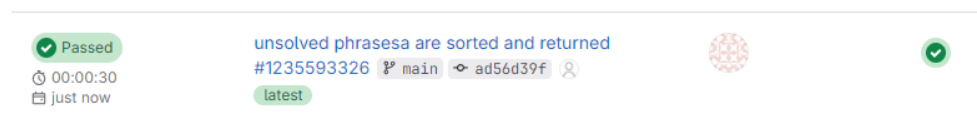
CI Testing

As mentioned earlier, continuous integration (CI) testing was employed for each of the three web services, with a specific focus on the backend function, which underwent both a build and test phase. These tests were instrumental in identifying and tracking any potential breaking changes being pushed to the GitLab repository. By maintaining separate repositories for each service, the platform facilitated swift analysis of any issues arising from failed or game-breaking tests. This proactive approach to CI testing ensured the early detection of errors or inconsistencies, allowing for timely resolution and maintaining the overall stability and reliability of the puzzle-solving platform.

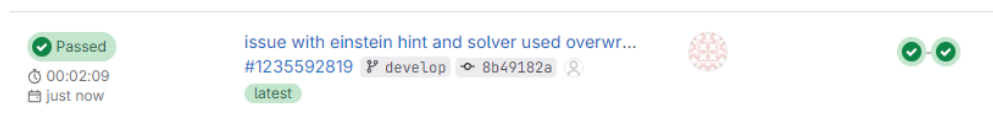
Frontend.



Python Script



Backend



Testing Screenshot - Appendix C

issues and bugs were uncovered solely through the CI pipeline, which remained undetected in the development environment. CI testing played a pivotal role in ensuring the delivery of high-quality, error-free code for the puzzle-solving platform. Its significance was profound throughout the platform's development, as it provided invaluable feedback on code changes and enhancements. By continuously integrating and testing code, the CI pipeline helped maintain the platform's reliability and stability, ultimately contributing to its overall success.

Evaluation

From the extensive testing conducted on the puzzle-solving platform, it has been demonstrated that the platform effectively meets its essential criteria of being able to solve the two sets of puzzles provided. To further evaluate its effectiveness in generating and solving these puzzles, a performance analysis will be conducted. This analysis will involve running the generation and solving processes five times for each puzzle type, including Sudoku puzzles of various grid sizes and the Einstein puzzle.

For Sudoku puzzles, the generation and solving processes will be performed separately for each grid size (2x2, 4x4, 9x9, and 16x16). By running these processes multiple times, an average generation time and solving time can be calculated for each grid size. This will provide insight into the platform's efficiency in handling Sudoku puzzles of different complexities and sizes.

As for the Einstein puzzle, a single overall generation and solving time will be measured since the initial phrases are already solved upon generation. This approach accounts for the unique nature of the Einstein puzzle and allows for a fair assessment of the platform's performance in generating and solving this type of puzzle.

By analysing the average generation and solving times for both Sudoku and Einstein puzzles, a comprehensive understanding of the platform's efficiency and effectiveness in puzzle generation and solving can be obtained. This performance analysis will further validate the platform's capability to meet user expectations and provide a seamless puzzle-solving experience.

Results

Performance Analysis

Sudoku Performance Results

Generation Results

Grid Size	1st	2nd	3rd	4th	5th	Average
2x2	6.60/ms	5.40/ms	4.90/ms	5.00/ms	5.70/ms	5.52/ms
4x4	6.80/ms	5.30/ms	4.80/ms	5.30/ms	4.90/ms	5.42/ms
9x9	6.70/ms	4.90/ms	5.70/ms	5.20/ms	5.40/ms	5.58/ms
16x16	5.80/ms	6.10/ms	5.20/ms	6.00/ms	5.00/ms	5.62/ms

Performance Results - Appendix F

Solving Results

Grid Size	1st	2nd	3rd	4th	5th	Average
2x2	4.60/ms	4.60/ms	5.00/ms	5.10/ms	4.80/ms	4.82/ms
4x4	6.80/ms	4.60/ms	5.10/ms	4.50/ms	4.20/ms	5.04/ms
9x9	6.60/ms	5.30/ms	10.00/ms	7.30/ms	8.10/ms	7.46/ms
16x16	129.10/ms	3,451.20/s m	4,731.70/m s	193.80/ms	6,722.50/m s	3,045.66/m s

Performance Results - Appendix F

Sudoku Results Analysis

The generation results for the Sudoku puzzles reveal interesting insights into the platform's efficiency in creating puzzles of varying complexities. Across different grid sizes (2x2, 4x4, 9x9, and 16x16), the average generation times exhibit remarkable consistency, indicating the effectiveness of the algorithm employed.

Upon examining the data, it's evident that the average generation times for each grid size are relatively similar. For instance, the average generation time for the 2x2 grid is approximately 5.52

milliseconds, while for the 4x4 grid, it's around 5.42 milliseconds. Similarly, the average generation times for the 9x9 and 16x16 grids are approximately 5.58 milliseconds and 5.62 milliseconds, respectively.

This consistent pattern of generation times suggests that the platform's algorithm for generating Sudoku puzzles efficiently adapts to the grid's size and complexity. One possible explanation for this consistency is the utilisation of a randomised approach in initialising the puzzles. By randomly selecting a quarter of the grid spaces and populating them with assorted initial numbers, the algorithm effectively kick-starts the puzzle generation process. This randomization ensures that each puzzle's initial state is sufficiently diverse, leading to a balanced and varied set of puzzles.

Moreover, the response time for puzzle generation remains relatively stable across different grid sizes, further indicating the efficiency of the algorithm. The similar average generation times across various grid sizes highlight the scalability and adaptability of the platform's generation algorithm. Whether generating a small 2x2 puzzle or a larger 16x16 puzzle, the platform consistently delivers puzzles within a comparable timeframe, showcasing its reliability and effectiveness in puzzle creation.

Overall, the generation analysis underscores the platform's capability to efficiently generate Sudoku puzzles of varying complexities while maintaining consistent response times. The similarity in average generation times across different grid sizes reflects the effectiveness of the randomised initialization approach and highlights the platform's reliability in delivering diverse and challenging puzzles to users.

The analysis of solving times for Sudoku puzzles reveals a notable trend of gradual increase in solving time with the grid size, culminating in a dramatic spike for the 16x16 grid. This pattern suggests that the solving algorithm, likely a backtracking algorithm, faces escalating challenges as the puzzle complexity and grid size increase.

For the smaller grid sizes of 2x2 and 4x4, the solving times remain relatively consistent and manageable. However, as the grid size expands to 9x9, there's a discernible uptick in solving time, indicating a slightly more intricate solving process. This increase in solving time can be attributed to the larger number of grid spaces and initial numbers, necessitating more iterations and validations by the solving algorithm.

The most striking observation emerges with the 16x16 grid, where the solving time experiences a drastic escalation compared to the smaller grid sizes. This substantial increase in solving time is likely attributable to several factors. Firstly, the backtracking algorithm employed for solving Sudoku puzzles may struggle to navigate through the larger dataset and complex ruleset inherent in a 16x16 grid. With 256 total grid spaces and 64 prepopulated initial numbers, the puzzle presents a significantly more intricate problem for the algorithm to unravel.

Furthermore, the exponential increase in possibilities and constraints within a 16x16 grid imposes a greater computational burden on the solving algorithm. Unlike the smaller grid sizes, which have fewer grid spaces and initial numbers, the 16x16 grid introduces a multitude of potential solutions and stricter rule sets, exacerbating the complexity of the solving process.

While the progression from smaller to larger grid sizes in Sudoku puzzles follows a logical trajectory, the leap to a 16x16 grid underscores the substantial computational challenges inherent in scaling up the puzzle size. Despite the algorithm's robustness, the 16x16 grid presents a formidable puzzle-solving task, resulting in a markedly longer solving time compared to its smaller counterparts.

In conclusion, the analysis of solving times highlights the escalating computational complexity associated with larger Sudoku grid sizes, particularly evident in the substantial increase in solving time for the 16x16 grid. This underscores the intricate nature of puzzle-solving algorithms and the computational challenges they encounter when confronted with larger and more complex problem domains.

The Sudoku puzzle-solving platform demonstrates commendable performance, with efficient generation times and reasonably manageable solving times for smaller grid sizes. While the platform effectively scales to larger grid sizes, as evidenced by the consistent generation times, the substantial increase in solving time for the 16x16 grid highlights the inherent computational challenges associated with larger Sudoku puzzles. Nevertheless, the platform's ability to generate and solve Sudoku puzzles across varying grid sizes underscores its reliability, effectiveness, and suitability for providing users with engaging and challenging puzzle-solving experiences.

In conclusion, the Sudoku puzzle-solving platform exhibits robust performance across different grid sizes, with efficient generation and reasonably manageable solving times. While larger grid sizes pose

computational challenges, the platform demonstrates commendable scalability and reliability, reaffirming its efficacy in delivering engaging and challenging Sudoku puzzles to users.

For the solving results

Einstein Performance Results

Solving and Generation Time

Iteration	Generation Time	Solving Time	Total
1st	3,371.20/ms	8.80/ms	3380.00/ms
2nd	5,412.70/ms	6.50/ms	5419.20/ms
3rd	1098.20/ms	6.40/ms	1104.40/ms
4th	4611.10/ms	6.00/ms	4617.10/ms
5th	3768.70/ms	6.20/ms	3774.90/ms
Average Time	3652.38/ms	6.78/ms	
Average Solving/Generation Time			3659.12/ms

Performance Results - Appendix F

Einstein Results Analysis

The performance analysis of the Einstein puzzle-solving platform provides valuable insights into the efficiency and effectiveness of the platform's generation and solving processes.

The generation time for the Einstein puzzle is notably longer compared to Sudoku puzzles, because not just the creation of the phrases but the initial sorting of phrases to houses and owners happens during the generation process. The average generation time of approximately 3652.38/ms indicates the time taken for the platform to generate puzzles by creating the phrases and then assigning phrases to houses and owners accurately. Despite the longer load time, the platform demonstrates efficiency in generating Einstein puzzles, ensuring the appropriate distribution of phrases and maintaining puzzle integrity.

In contrast to Sudoku puzzles, where the solving process heavily relies on backtracking algorithms to navigate through the puzzle space and deduce the correct values for each cell, the Einstein puzzle-solving process exhibits less reliance on backtracking alone. While backtracking remains a crucial component of the solving algorithm, its role is supplemented by other strategies and heuristics due to the nature of the puzzle.

Unlike Sudoku, where each cell must adhere to strict rules and constraints based on row, column, and box, the Einstein puzzle-solving process involves assigning phrases to houses and owners while ensuring consistency and logic using the developed python script. This difference in problem structure results in a less complex puzzle space and fewer constraints, reducing the computational burden on the backtracking algorithm.

Additionally, since the phrases are pre-sorted during puzzle generation via the NLP script called during the generation phase, the solving algorithm can leverage this initial arrangement to narrow down possibilities and make informed decisions. Strategies such as constraint propagation, inference, and intelligent guessing play significant roles in efficiently solving Einstein puzzles.

As a result, while backtracking remains an essential technique in the solving process, its workload is alleviated by the puzzle's structure and additional solving strategies. This reduced reliance on backtracking contributes to shorter solving times and less computational complexity compared to Sudoku iterations, making the Einstein puzzle-solving experience more streamlined and efficient.

The Einstein puzzle-solving platform demonstrates commendable performance, with efficient generation and solving times. Despite the longer load time associated with generation, the platform efficiently distributes phrases to houses and owners, ensuring puzzle integrity and complexity. The solving process, facilitated by the backtracking algorithm, effectively deduces missing information and completes puzzles accurately. Overall, the platform's performance underscores its reliability, efficiency, and effectiveness in providing users with engaging and challenging Einstein puzzles.

In conclusion, the Einstein puzzle-solving platform exhibits robust performance in both generation and solving processes, ensuring the creation of challenging puzzles and accurate solutions. The platform's efficiency and accuracy highlight its suitability for delivering engaging puzzle-solving experiences to users.

Comparative Analysis

For the comparative analysis, a brief study was undertaken involving five participants tasked with solving the same Sudoku grids and Einstein puzzle provided above. Additionally, an online Sudoku solver was utilised to benchmark the results against both the developed puzzle solver employed in this study and traditional paper-based Sudoku solving methods. However, due to the absence of an online Einstein puzzle solver, the results were compared solely against solutions derived from conventional paper-based approaches used by participants. This multifaceted approach aimed to provide insights into the effectiveness and efficiency of the developed puzzle solver in relation to both digital and traditional solving methods, offering a comprehensive assessment of its performance and usability.

Paper based approach

Participant	Sudoku 2x2	Sudoku 4x4	Sudoku 9x9	Sudoku 16x16	Einstein Puzzle
Person 1	< 1 sec	32 sec	18 min	> 1 hour	2 min 37 sec
Person 2	< 1 sec	37 sec	25 min	> 1 hour	1 min 40 sec
Person 3	< 1 sec	50 sec	29 min	> 1 hour	2 min 15 sec
Person 4	< 1 sec	29 sec	20 min	> 1 hour	1 min 59 sec
Person 5	< 1 sec	35 sec	26 min	> 1 hour	2 min 27 sec

Performance Results - Appendix F

Sudoku Online Solver

[7] <https://sudokuspoiler.com/sudoku/sudoku9>

Grid size	1st	2nd	3rd	4th	5th	Average
9x9	132 ms	137 ms	139 ms	135 ms	139 ms	136.4 ms
16x16	407 ms	498 ms	501 ms	424 ms	467 ms	459.4 ms

Performance Results - Appendix F

The comparative analysis between paper-based solving, online solvers, and the developed puzzle solver reveals notable differences in performance across different grid sizes and puzzle types.

In the paper-based approach, participants exhibited varying solving times across Sudoku grid sizes and the Einstein puzzle. Notably, for the Sudoku puzzles, participants generally solved the smaller 2x2 and 4x4 grids within seconds, while the larger 9x9 and 16x16 grids required significantly more time, with some participants taking over an hour to complete the 16x16 grid. For the Einstein puzzle, solving times were generally shorter, with participants completing it within a few minutes, indicating its relatively lower complexity compared to larger Sudoku grids.

On the other hand, the online Sudoku solver provided consistent and quick solutions, especially for the 9x9 grid, with an average solving time of 136.4 milliseconds. However, for the 16x16 grid, the solving time increased significantly, averaging 459.4 milliseconds per iteration. It's worth noting that the online solver was unavailable for 2x2 and 4x4 grids, limiting the comparison.

In contrast, the developed puzzle solver demonstrated competitive performance across Sudoku grid sizes, with generally quick solving times. However, a significant discrepancy was observed in the solving time for the 16x16 grid, which exhibited a dramatic increase compared to smaller grids. While the solver efficiently handled smaller grids, the complexity of the 16x16 Sudoku grid posed challenges, resulting in considerably longer solving times.

Overall, while the developed puzzle solver proved effective for most grid sizes, further optimization is required to enhance its performance, particularly for larger grids like the 16x16 Sudoku. The online solver's consistent performance across grid sizes highlights the potential for improvement in the developed solver to achieve faster solving times, especially for more complex puzzles.

In some variable cases, the developed system took about 19 to 25 seconds to solve the 16x16 grid, significantly longer than the ideal solving time observed for smaller grids. This prolonged solving time underscores the inherent complexity of larger Sudoku grids, which contain a greater number of cells and possible combinations, thereby posing challenges for the solving algorithm to navigate through the solution space efficiently. The discrepancy in solving times between smaller and larger grids suggests that further optimization of the solving algorithm is necessary to address the computational complexity inherent in larger Sudoku puzzles. By refining the algorithm to handle larger grids more effectively, the developed system can achieve more consistent and competitive performance across a broader range of puzzle sizes.

Discussion

Interpretation of Results

The results obtained from the performance analysis of both Sudoku and Einstein puzzle solvers provide valuable insights into the effectiveness and limitations of the developed systems. In Sudoku, the generation times remained relatively consistent across different grid sizes, indicating that the algorithm used for generating puzzles is efficient and scalable. However, the solving times showed a significant increase for larger grid sizes, particularly the 16x16 grid, highlighting the computational complexity associated with solving larger puzzles. This underscores the need for further optimization of the solving algorithm to improve performance on larger grids.

Similarly, the Einstein puzzle solver exhibited consistent solving times across multiple iterations, suggesting that the backtracking algorithm used is effective in solving the puzzle. However, the lengthy generation times indicate a potential area for improvement, as the initial sorting of phrases to houses and owners upon generation consumes considerable computational resources.

Challenges and Limitations

In the context of Sudoku, the challenges primarily revolved around the complexity of working with larger grid sizes and the computational resources required to generate and solve puzzles efficiently. While the initial plan was to develop a dynamic solver capable of handling grids of various sizes seamlessly, the implementation of such a system proved challenging, particularly for the 16x16 grid. The exponential increase in the number of cells and possible combinations necessitated a more structured approach to algorithm design and optimization. Additionally, the reliance on backtracking as the primary solving technique posed limitations in terms of scalability and efficiency, especially when confronted with the intricate rules and constraints of larger grids. Despite efforts to optimise the solving algorithm, the computational complexity inherent in larger grid sizes remained a significant hurdle, leading to longer solving times and decreased solver performance.

In the case of the Einstein puzzle, integrating Natural Language Processing (NLP) techniques presented its own set of challenges. While the initial plan was to develop a dynamic solver capable of parsing textual clues and deriving meaningful constraints for solving the logic problem, the complexity of working with spaCy's NLP capabilities required a more structured approach. The advanced linguistic processing capabilities of spaCy enabled the extraction of key information from

textual clues, such as nationalities, colours, and pets, but implementing these capabilities into a cohesive solver proved to be a non-trivial task. Custom rule-based matching and dependency parsing were essential components in formulating specific constraints based on linguistic patterns observed in the puzzle clues. However, integrating these techniques into the solver architecture required careful consideration of data structures, algorithmic complexity, and computational efficiency. Despite the challenges, the integration of NLP techniques, particularly through the utilisation of spaCy, significantly enhanced the efficiency and effectiveness of solving the Einstein puzzle within the digital framework, showcasing the potential for advanced linguistic processing in puzzle-solving applications.

Another significant challenge emerged during the project, particularly concerning the integration of Jest as an automated front-end testing suite. Despite initial plans to incorporate Jest to automate front-end testing processes, the complexity of this task exceeded the time constraints imposed by the assignment's due date. As a result, the team opted to focus on developing in-depth manual test cases instead, ensuring thorough testing of the developed system within the available time frame. While the decision to forego automated testing was a compromise necessitated by time constraints, the comprehensive manual testing approach effectively validated the system's functionality and reliability across various scenarios.

Overall, the development of the puzzle-solving system encountered several challenges and obstacles. However, through quick thinking and resourcefulness, viable alternatives were identified and implemented to overcome these hurdles and meet the project's deadline. Despite the unexpected issues, adaptability and resilience were key, ultimately resulting in the delivery of a cohesive and robust system capable of dynamically generating and solving puzzles.

Implications of Practice and Research

The development of a robust puzzle-solving system capable of dynamically accommodating future puzzles, such as crosswords, underscores the importance of adaptability and reliability in software design. By implementing a flexible architecture that can seamlessly integrate new puzzle types, developers can future-proof their systems and ensure long-term viability. This practice not only enhances the user experience by offering a diverse range of puzzles but also minimises the need for extensive redevelopment efforts when introducing new features. It serves as a valuable lesson for developers, highlighting the significance of scalability and extensibility in software engineering projects.

In the realm of research, leveraging Natural Language Processing (NLP) to solve complex logic puzzles presents intriguing possibilities and challenges. While the application of NLP techniques introduces a layer of complexity, particularly in parsing textual clues and deriving meaningful constraints, it offers unprecedented opportunities for enhancing puzzle-solving algorithms. Through advanced linguistic processing and rule-based matching, NLP enables the extraction and interpretation of crucial information from puzzle clues, facilitating more accurate and efficient solutions. Despite the inherent difficulty in implementing NLP for puzzle-solving, research in this area holds promise for advancing the capabilities of artificial intelligence in tackling complex logic problems. Further exploration and experimentation are warranted to refine NLP-based approaches and unlock their full potential in puzzle-solving domains.

The development and research efforts showcased in this study underscore the importance of innovation and adaptability in software engineering and artificial intelligence. By creating a robust puzzle-solving system capable of dynamically handling various puzzle types, developers demonstrate the value of flexible and scalable architectures in meeting evolving user demands. Moreover, the exploration of Natural Language Processing (NLP) techniques for solving complex logic puzzles highlights the potential of interdisciplinary approaches in advancing AI capabilities. As technology continues to evolve, embracing innovative methodologies and research avenues will be essential for driving progress and achieving breakthroughs in software development and artificial intelligence. Through continuous exploration and collaboration, developers and researchers alike can push the boundaries of what is possible, ultimately enhancing the quality and effectiveness of digital solutions for users worldwide.

Conclusion

Summary

The paper presents the development and analysis of a puzzle-solving algorithm capable of dynamically generating and solving Sudoku and Einstein puzzles. Through rigorous testing and performance analysis, the effectiveness and limitations of the developed systems were evaluated. The results indicate consistent generation times for Sudoku puzzles across different grid sizes, highlighting the algorithm's efficiency and scalability. However, solving times increased significantly for larger grid sizes, posing challenges in algorithm optimization, particularly for the 16x16 grid. In contrast, the Einstein puzzle solver exhibited consistent solving times but faced challenges in lengthy

generation times due to the initial parsing of textual clues using Natural Language Processing (NLP) techniques.

Recommendations

To enhance the puzzle-solving algorithm's capabilities and accommodate future puzzles, several recommendations are proposed. Firstly, for Sudoku puzzles, further optimization of the solving algorithm, especially for larger grid sizes, is necessary to reduce solving times and improve performance. Exploring alternative solving techniques beyond backtracking may yield faster and more efficient solutions, particularly for complex puzzles like the 16x16 grid. Additionally, for the Einstein puzzle, improving the efficiency of NLP techniques, such as spaCy, could streamline the parsing and constraint derivation process, reducing generation times and enhancing solver performance. Furthermore, considering the challenges encountered in integrating Jest for automated front-end testing, future efforts should focus on finding more feasible solutions or alternatives to streamline testing processes.

Reflection

Overview

The development and analysis of the puzzle-solving algorithm provided valuable insights into the complexities and challenges inherent in solving Sudoku and Einstein puzzles. One of the key takeaways from the project is the importance of continuously evaluating and optimising solving algorithms to improve performance and scalability, particularly for larger and more complex puzzles. The challenges encountered, such as the sluggish performance of the backtracking algorithm on the 16x16 grid and the intricacies of integrating NLP techniques, underscore the need for innovative solutions and interdisciplinary approaches in puzzle-solving domains. Moving forward, addressing these challenges through alternative solving techniques and enhanced utilisation of NLP capabilities will be crucial for advancing the algorithm's effectiveness and efficiency. Additionally, leveraging lessons learned from testing and performance analysis, future iterations of the puzzle-solving algorithm can benefit from more robust optimization strategies and streamlined development processes. Overall, the project highlights the iterative nature of software development and the importance of adaptability and innovation in tackling complex problem-solving tasks.

Reflection on Research Objectives

Throughout the development and evaluation of the puzzle-solving web application, several research objectives were pursued, each with varying degrees of success.

1. **Design and Development of User-Friendly Web Application (Met):** The objective to create a user-friendly web application for generating and solving Sudoku puzzles was successfully achieved. The application featured an intuitive interface that allowed users to interact seamlessly with puzzles, providing a positive user experience.
2. **Implementation of Efficient Solving Algorithms (Met with Exceptions):** While efficient solving algorithms were implemented for both Sudoku and Einstein puzzles, there were exceptions. The algorithms proved effective for standard Sudoku puzzles, but challenges arose with larger puzzle sizes, particularly the 16x16 grid. The solving algorithm struggled to maintain efficiency on the larger grid, indicating room for further optimization.
3. **Evaluation of Performance and Usability (Partially Met):** Usability testing and performance evaluations were conducted to assess the web application's effectiveness and efficiency. While the evaluations provided valuable insights into user satisfaction and overall performance, there were areas where improvements could have been made to enhance usability further.
4. **Comparison of Solving Algorithms (Met):** The effectiveness of different solving algorithms was compared within the web application, providing valuable data on their efficacy in solving Sudoku and Einstein puzzles under various scenarios. This objective was successfully achieved, offering insights into the strengths and limitations of each algorithm.
5. **Exploration of Potential Applications (Partially Met):** While the exploration of potential applications beyond entertainment was initiated, there is room for improvement. Specifically, the implementation of Natural Language Processing (NLP) techniques to enhance the practical applications of the web application requires further development. More robust utilisation of NLP could unlock additional educational, research, and practical applications beyond the platform.
6. **Identification of Challenges and Limitations (Met):** Challenges and limitations encountered during the development process were identified and addressed, reflecting a commitment to iterative improvement. Technical constraints, algorithmic complexities, and user interface design considerations were all taken into account, contributing to the refinement of the puzzle-solving web application.

In summary, while many of the research objectives were successfully met, there remain areas for improvement and future exploration. Addressing challenges related to algorithm efficiency, usability

enhancements, and NLP implementation will be crucial for advancing the capabilities and practical applications of the puzzle-solving web application in diverse contexts.

Reflection on Requirements

The puzzle-solving platform successfully achieved its core function definitions, demonstrating its ability to generate and solve puzzles as intended. The user interface was developed with intuitiveness in mind, providing users with a seamless experience. However, the integration of text analysis using SpaCy NLP left room for improvement, requiring more time to fully leverage the capabilities of SpaCy in identifying key entities and creating relationships from them.

All functional requirements were met by the developed system. It effectively generated and solved Sudoku and Einstein puzzles of varying difficulty levels, with the exception of difficulty settings for Einstein due to time constraints. User input functionality was successfully implemented for all puzzles, providing accurate returns of information. Hint options were available for both puzzle types, and the user interface displayed all required information while ensuring a smooth user experience.

The system performed optimally for most puzzles, with acceptable generation times and scalability. However, the complexity of the 16x16 Sudoku grid led to longer solve times, highlighting an area for potential optimization. The platform's architecture demonstrated scalability, allowing for the seamless incorporation of additional puzzles without the need for extensive rewriting. The user interface met requirements for responsiveness and intuitive controls, leveraging React JS to create dynamic components and pages for a user-friendly experience. Additionally, the system proved to be robust and reliable, minimising errors and downtime during the puzzle generation and solving process.

These achievements demonstrate the platform's effectiveness in meeting its defined requirements, with opportunities for further improvement identified in specific areas such as text analysis and performance optimization.

References

Frontend Git Repository: https://gitlab.com/c_mone03/puzzle-generator-and-solver

Backend Git Repository: https://gitlab.com/c_mone03/puzzle-generator-and-solver-api

Python Web Service Git Repository: https://gitlab.com/c_mone03/puzzle-generator-and-solver-nlp

[1] Lijo, V. P., and Jasmin T. Jose. "Solving N-Queen Problem by Prediction." *Int. J. Comput. Sci. Inf. Technol* 6 (2015): 3844-3848.

[2] Maji, Arnab Kumar, and Rajat Kumar Pal. "Sudoku solver using minigrid based backtracking." *2014 IEEE International Advance Computing Conference (IACC)*. IEEE, 2014.

[3] Chowdhary, KR1442, and K. R. Chowdhary. "Natural language processing." *Fundamentals of artificial intelligence* (2020): 603-649.

[4] Fisher, Ronald. "Fisher-Yates shuffle."

[5] Delahaye, Jean-Paul. "The science behind Sudoku." *Scientific American* 294.6 (2006): 80-87.

[6] Patterson, Mike C., Bob Harmel, and Dan Friesen. "A spreadsheet solution to Einstein's Riddle." *International Journal of Technology, Policy and Management* 7.1 (2007): 49-67.

[7] <https://sudokuspoiler.com/sudoku/sudoku9>

Appendices

User Manual

To run the project please download all the development code from the 3 github repositories linked in References, each set of code is essential for running the document and they are split into 3 repositories to ensure simplicity. Please run all steps in order.

Starting the python web service:

Step 1. Download the code from the repository for python web service.

Step 2. Ensure Docker is installed on your machine. And have docker desktop open.

<https://www.docker.com/products/docker-desktop/>

Step 3. Open a Git Bash terminal where the python code is stored

Step 4. Run this command

```
Conall Mone@DESKTOP-6NM5V0I MINGW64 ~/Documents/2023-Computer_Science_Project/Py
thon NLP (main)
$ docker build -t einstein_puzzle .|
```

Step 5. Then run this command to run the einstein puzzle solving script

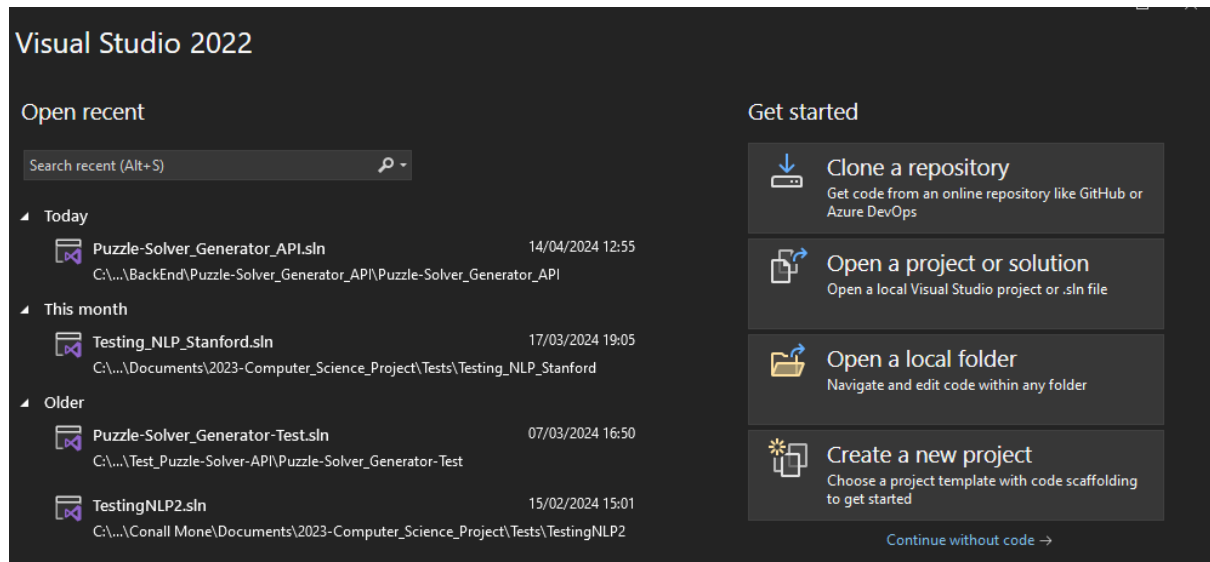
```
Conall Mone@DESKTOP-6NM5V0I MINGW64 ~/Documents/2023-Computer_Science_Project/Py
thon NLP (main)
$ docker run -d -p 32769:5000 einstein_puzzle
```

Starting Backend:

Step 1. Download backend code from the repository for backend.

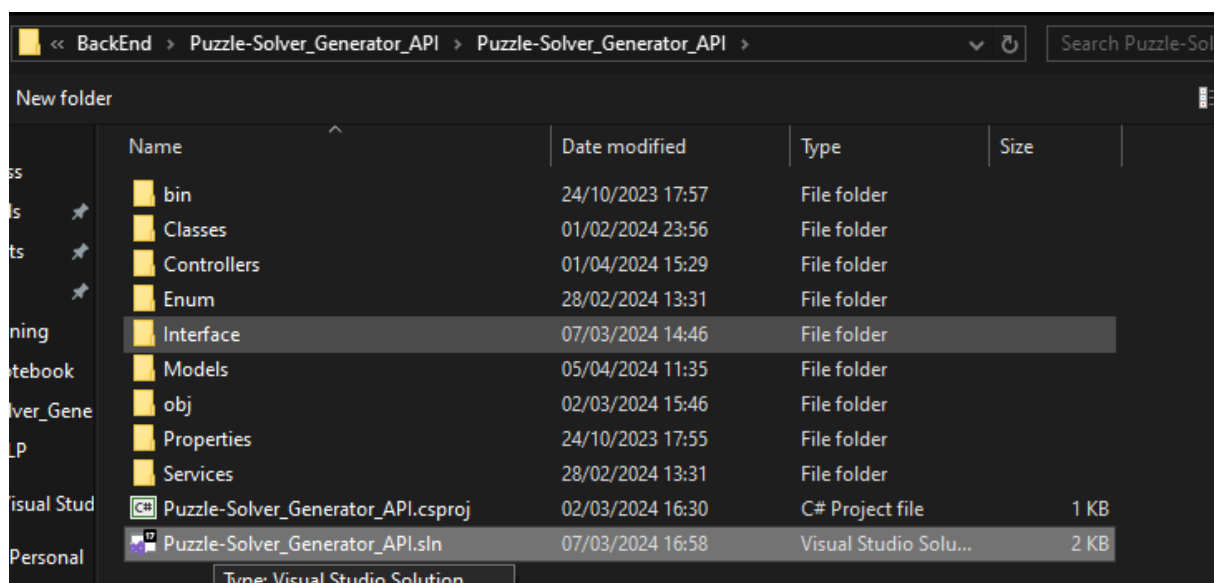
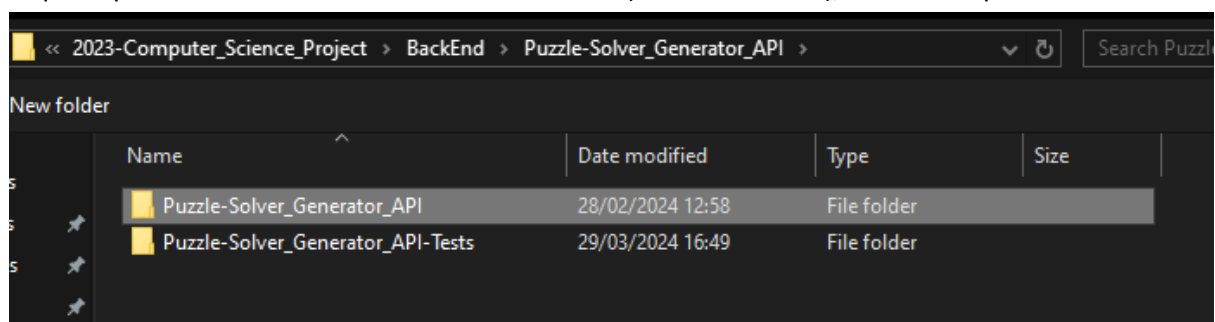
Step 2. Open visual studio

Step 3. Click open project or solution

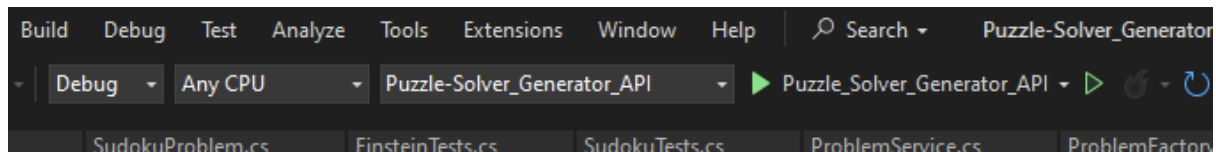


Step 4. Browse to the folder where code was cloned to then open the Puzzle-Solver_Generator-API folder

Step 5. Open the Puzzle Solver Generator API folder (not the test one), then click open on the .sln file



Step 6. Click the start button to run the application



Step 7. A swagger page will open with controller functions displayed. Minimise but do not leave or close this tab and Continue onto the other steps.

Starting the Frontend:

Step 1. Download the frontend code from the repository for Frontend.

Step 2. Open visual studio code.

Step 3. Click File >> Open Folder

Step 4. Browse to location frontend code was downloaded and open folder

Step 5. Run “npm install” in terminal (View >> Terminal)

```
Conall Mone@DESKTOP-6NM5V0I MINGW64 ~/Documents/2023-Computer_Science_Project/FrontEnd/puzzle_solver_generator (main)
$ npm install
```

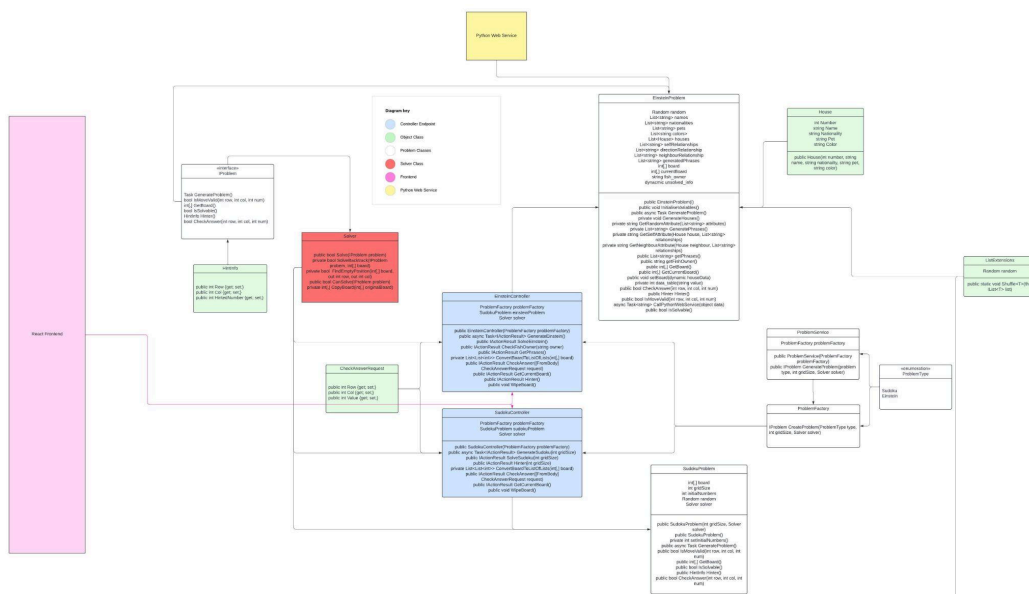
Step 6. Then Run “npm start” in the terminal.

```
Conall Mone@DESKTOP-6NM5V0I MINGW64 ~/Documents/2023-Computer_Science_Project/FrontEnd/puzzle_solver_generator (main)
$ npm start
```

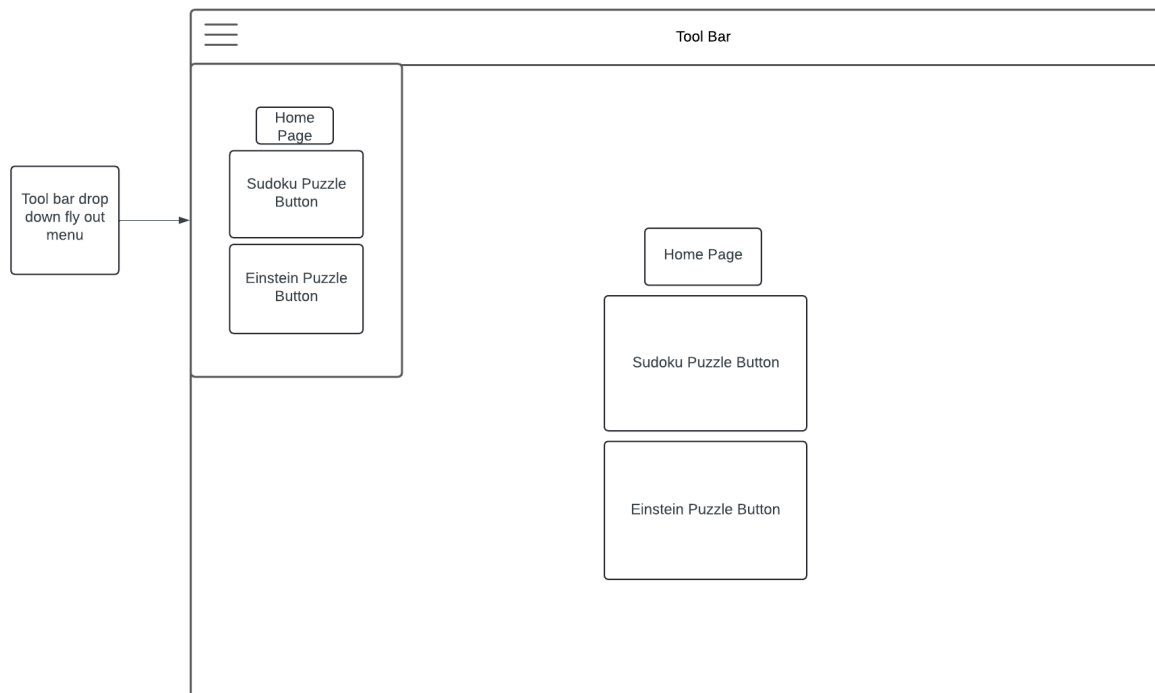
Step 7. The project will open a window to the homepage of the puzzle solver application.

Appendix A

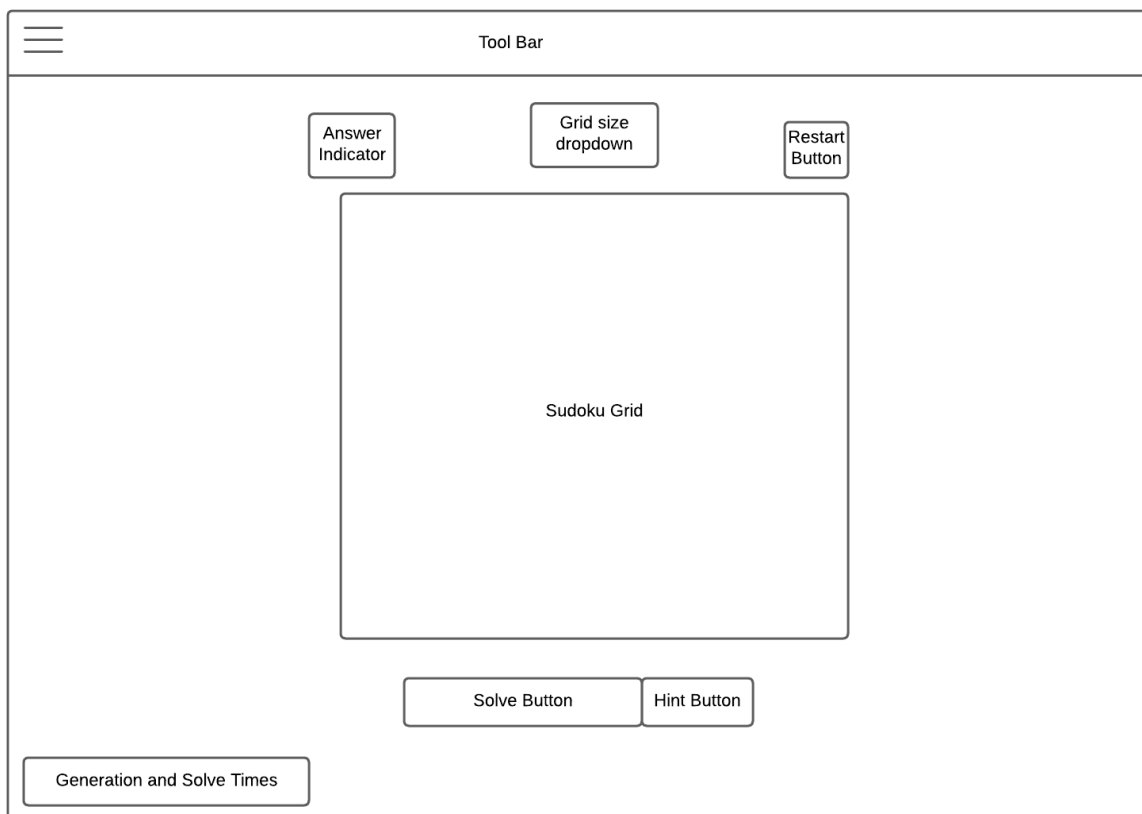
https://lucid.app/lucidchart/invitations/accept/inv_00cadcfef9ab474db30847314aa2d575



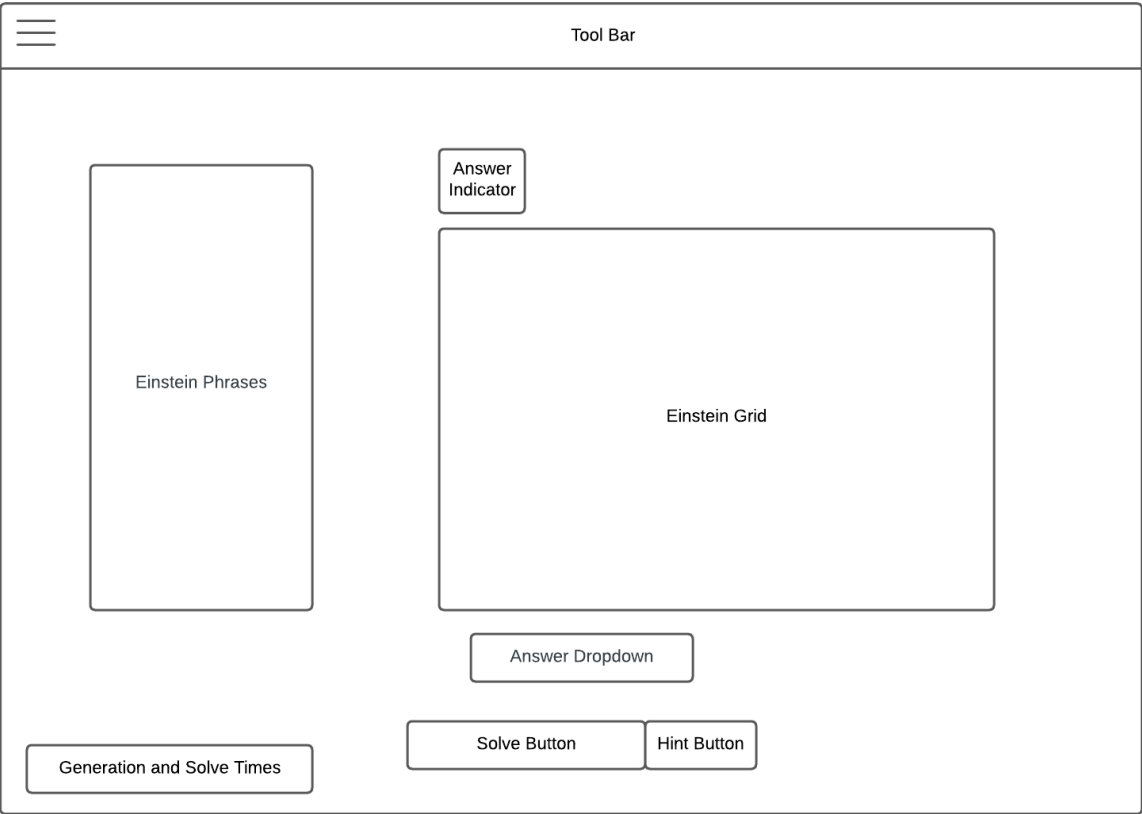
Appendix B



https://lucid.app/lucidchart/invitations/accept/inv_dfe51495-fa46-4e13-8fe1-3b6d30d71432



https://lucid.app/lucidchart/invitations/accept/inv_08d9f253-3af0-49b7-b360-68961ddb8b78



Appendix C

✓ EinsteinTest (6)	8.2 sec
✓ SudokuTest (23)	18 sec
✓ CheckAnswer_IncorrectAnswer_...	211 ms
✓ CheckAnswer_ReturnsBadRequ...	2 ms
✓ Hinter_ReturnsBadRequest	< 1 ms
✓ GenerateSudoku_Valid (4)	< 1 ms
✓ GetCurrentBoard_ReturnsBadRe...	< 1 ms
✓ GetCurrentBoard_Valid (4)	< 1 ms
✓ HinterSudoku_ValidProblem (4)	1 ms
✓ SolveSudoku_SolvableProblem (..	17.8 sec
✓ SolveSudoku_ReturnsBadRequest	< 1 ms

✓ EinsteinTest (8)	8.2 sec
✓ CheckAnswer_ReturnsBadRequ...	10 ms
✓ CheckFishOwner_ReturnsBadRe...	< 1 ms
✓ CheckFishOwner_ReturnsIncorr...	4.2 sec
✓ GetCurrentBoard_ReturnsBadRe...	< 1 ms
✓ GetPhrases_ReturnsBadRequest	< 1 ms
✓ GetPhrases_Valid	4.1 sec
✓ Hinter_ReturnsBadRequest	< 1 ms
✓ SolveEinstein_ReturnBadRequest	< 1 ms

✓ Passed 🕒 00:01:14 📅 just now	issue with double digits required on sudoku gri... #1235592313 👤 main 🔗 83bc65ce 👤 latest	 ✓
✓ Passed 🕒 00:00:30 📅 just now	unsolved phrasesa are sorted and returned #1235593326 👤 main 🔗 ad56d39f 👤 latest	 ✓
✓ Passed 🕒 00:02:09 📅 just now	issue with einstein hint and solver used overwr... #1235592819 👤 deveLop 🔗 8b49182a 👤 latest	 ✓ ✓

Appendix D

Meeting Minutes

Meeting Minutes 9/10/23

CM Conall Mone To: Iain Styles	😊 ↩ Reply ↩ Reply al
Create puzzle solver using constraints Start with sudoku and try to scale up to Einstein problem and possibly attempt crossword with natural language processing	
Sudoku — start small the bigger — React as front end and c# back end — Make code robust for reusability — Generating uses the constraints and random number allocation — Multiple answers very likely	
Einstein — needs natural language processing — Contract the language to make processing easier	
Crossword • requires heavy and difficult use of natural language processing • Generation of crossword drawing black and white squares for grid and dictionary words to assign questions and answers	
Specification — the code language being used — talk about the scaling of projects — Essential and desirable requirements	

Project Meeting Minutes 24/10/23

CM

Conall Mone

To: Iain Styles

- Discussed sending me references for constraints handling
 - Drew example and explained the constraints for sudoku on 4x4 grid
 - I will have a small demo of a player interactable sudoku
 - Possible involve a hint to use constraints to find possible answer
-

Meetings Minutes 05/01/24

CM

Conall Mone

To: Iain Styles

- Improved hunter functionality
- Added a way to input answers into sudoku
- Added unit testing
- Added ci pipeline
- Issues with 4x4 grid need to test more
- Add limit to number of hints to make more game like
- Add success screen once grid solved
- Arrange a demo meeting for next week between the 8th and 12th

Meeting minutes 08/04/24

CM

Conall Mone

To: Iain Styles

- update on Einstein problem. It is implanted into backend frontend will be done soon
- Spacey nlp in python. But script is too specific and will need to be generalised
- Sudoku unit tests and ci pipeline done
- Frontend testing and Einstein unit tests to be done
- Python script has git lab and ci pipelines

Meeting Minutes 27/03/2024



Conall Mone

To: Iain Styles

- showed final draft of sudoku and einstein problem
- major bulk of code completed
- to do small fixes on front end ui and improvements
- write dissertation draft for next meeting after easter
- prepare for demo to be scheduled for the 19th

Appendix E

Sudoku Test Cases

Test Case	Input	Expected Outcome	Pass/Fail
TC1 - change grid size	Select all the options in the grid size dropdown	The grid will change to the corresponding grid size	Pass
TC2 - valid correct	Valid number in grid	Grid updated to reflect answer input. Answer indicator will turn to green tick	Pass
TC3 - valid incorrect	Valid but incorrect number in grid	Grid unchanged. Answer indicator will turn to red cross	Pass
TC4 - invalid input beyond minimum	Enter 0 into grid	Grid unchanged. Answer indicator will turn to red cross	Pass
TC5 - invalid input beyond maximum	Enter a number 1 beyond grid size maximum	Grid unchanged. Answer indicator will turn to red cross	Pass
TC6 - use first hint	Click hint button	Number will display on grid in blue and a hint icon removed	Pass

TC7 - use second hint	Click hint button	Number will display on grid in blue and hint icon removed	Pass
TC9 - use third hint	Click hint button	Number will display on grid in blue and hint icon removed	Pass
TC10 - use fourth hint	Click hint button	Alert appear saying out of hints	Pass
TC11 - Solve button	Click the solve button	Grid will be completed and message will appear	Pass
TC12 - solve with hints	Click the hint button and then the solve button	Grid will be completed and message will appear	Pass
TC13 - manually solve	Manually input the answers to the grid	Grid will be completed and message will appear	Pass
TC14 - refresh button	Click the refresh button	Current selected grid size will refresh a new grid	Pass

Einstein Test Cases

Test Case	Input	Expected Outcome	Pass/Fail
TC1 - valid correct colour	Correct answer selected for colour	Grid updated to reflect answer input. Answer indicator will turn to green tick	Pass
TC2 - valid correct name	Correct answer selected for name	Grid updated to reflect answer input. Answer indicator will turn to green tick	Pass
TC3 - valid correct nationality	Correct answer selected for nationality	Grid updated to reflect answer input. Answer indicator will turn to green tick	Pass
TC4 - valid correct pet	Correct answer	Grid updated to reflect answer	Pass

	selected for pet	input. Answer indicator will turn to green tick	
TC5 - incorrect answer for colour	incorrect answer selected for colour	Grid unchanged. Answer indicator will turn to red cross	Pass
TC6 - incorrect name	Incorrect answer selected for name	Grid unchanged. Answer indicator will turn to red cross	Pass
TC7 - incorrect nationality	incorrect answer selected for nationality	Grid unchanged. Answer indicator will turn to red cross	Pass
TC8 - incorrect pet	incorrect answer selected for pet	Grid unchanged. Answer indicator will turn to red cross	Pass
TC9 - first hint option	Click hint button	The grid will display the correct answers on the grid	Pass
TC9 - second hint option	Click the hint button a second time	An alert will appear stating that you one hint is already used	Pass
TC10 - select correct fish owner	Select the correct fish owner from the select owner dropdown	A success message will display and a refresh button	Pass
TC11 - select correct fish owner	Select the incorrect fish owner from the select owner dropdown	A failed message will display and a refresh button	Pass
TC12 - house colour change	Select a valid house colour answer	The house icon above the selected column will change to the selected colour	Pass
TC13 - solve button	Click the solve button	The grid will populate and the correct fish owner will display with success message	Pass

TC14 - solve button with hints	Click the hint button then the solve button	The grid will populate and the correct fish owner will display with success message	Pass
TC15 - refresh button	Answer the fish owner dropdown then click the refresh button	The page will refresh	Pass

Appendix F

Sudoku Performance Results

Generation Results

Grid Size	1st	2nd	3rd	4th	5th	Average
2x2	6.60/ms	5.40/ms	4.90/ms	5.00/ms	5.70/ms	5.52/ms
4x4	6.80/ms	5.30/ms	4.80/ms	5.30/ms	4.90/ms	5.42/ms
9x9	6.70/ms	4.90/ms	5.70/ms	5.20/ms	5.40/ms	5.58/ms
16x16	5.80/ms	6.10/ms	5.20/ms	6.00/ms	5.00/ms	5.62/ms

Solving Results

Grid Size	1st	2nd	3rd	4th	5th	Average
2x2	4.60/ms	4.60/ms	5.00/ms	5.10/ms	4.80/ms	4.82/ms
4x4	6.80/ms	4.60/ms	5.10/ms	4.50/ms	4.20/ms	5.04/ms
9x9	6.60/ms	5.30/ms	10.00/ms	7.30/ms	8.10/ms	7.46/ms
16x16	129.10/ms	3,451.20/s m	4,731.70/m s	193.80/ms	6,722.50/m s	3,045.66/m s

Einstein Performance Results

Solving and Generation Time

Iteration	Generation Time	Solving Time	Total
1st	3,371.20/ms	8.80/ms	3380.00/ms
2nd	5,412.70/ms	6.50/ms	5419.20/ms
3rd	1098.20/ms	6.40/ms	1104.40/ms
4th	4611.10/ms	6.00/ms	4617.10/ms
5th	3768.70/ms	6.20/ms	3774.90/ms
Average Time	3652.38/ms	6.78/ms	
Average Solving/Generation Time			3659.12/ms

Paper based approach solve times

Participant	Sudoku 2x2	Sudoku 4x4	Sudoku 9x9	Sudoku 16x16	Einstein Puzzle
Person 1	< 1 sec	32 sec	18 min	> 1 hour	2 min 37 sec
Person 2	< 1 sec	37 sec	25 min	> 1 hour	1 min 40 sec
Person 3	< 1 sec	50 sec	29 min	> 1 hour	2 min 15 sec
Person 4	< 1 sec	29 sec	20 min	> 1 hour	1 min 59 sec
Person 5	< 1 sec	35 sec	26 min	> 1 hour	2 min 27 sec

Sudoku Online Solver times

Grid size	1st	2nd	3rd	4th	5th	Average
9x9	132 ms	137 ms	139 ms	135 ms	139 ms	136.4 ms
16x16	407 ms	498 ms	501 ms	424 ms	467 ms	459.4 ms